UNIVERSIDAD DEL VALLE

LI18 - Tercera práctica con Apache Hive

Materia: Big Data

Estudiante: Johnny Suarez Abasto

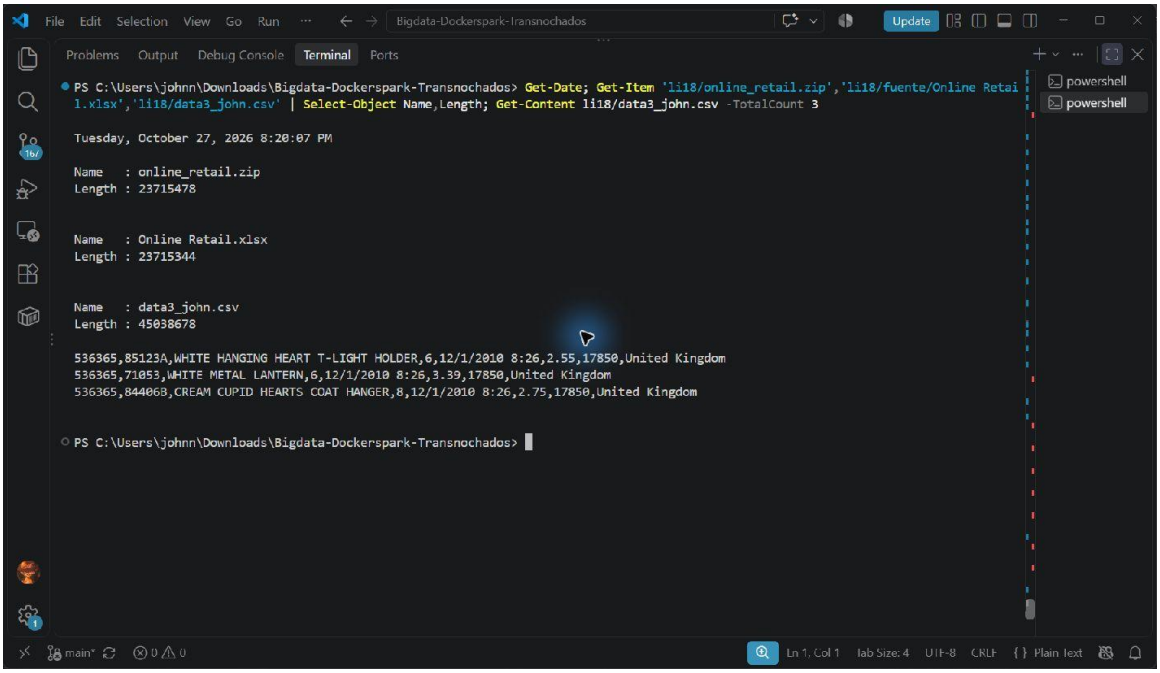
Grupo: 02

Fecha: 27 de octubre de 2026

Para este laboratorio, se asume que ya tiene Apache Hive corriendo:

1. Descargue el siguiente dataset: https://archive.ics.uci.edu/static/public/352/online+retail.zip

Se descargó online+retail.zip desde el repositorio UCI y se extrajo Online Retail.xlsx. El archivo contiene 541.909 registros y ocho columnas. Se conservó el Excel original.



Evidencia de los archivos originales y del CSV convertido en la terminal de VS Code.

2. La descarga anterior le presentará un archivo EXCEL denominado “Online Retail.xlsx”; deberá convertirlo a un CSV y renombrarlo como **data3_john.csv**

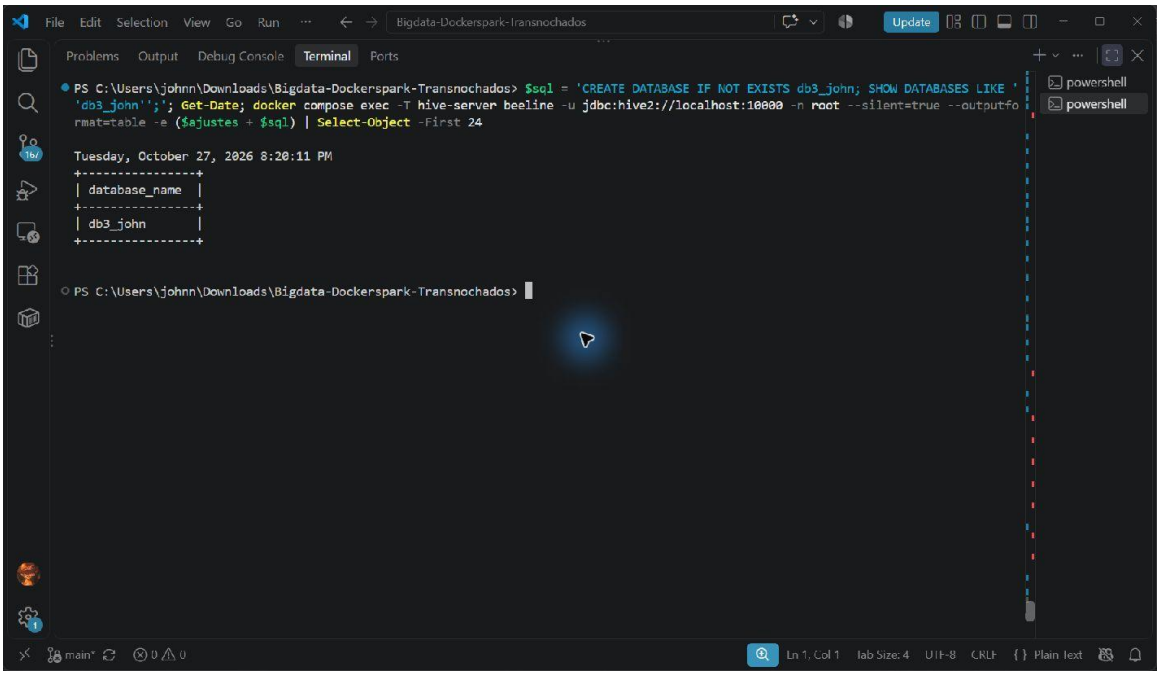
Se convirtió la hoja de Excel a data3_john.csv en UTF-8 y sin fila de encabezados. Las fechas se expresaron como M/d/yyyy H:mm para mantener compatibilidad con las consultas propuestas. Se conservaron las comillas de las descripciones, los precios cero y las cantidades negativas. Los identificadores de cliente vacíos se transformaron a NULL en la tabla final. El resultado se verificó con el conteo de 541.909 filas en Hive.

3. Acceder a Hive desde el contenedor **docker-hive_hive-server_1** con el comando: hive

Se utilizó el entorno del Grupo 02, Bigdata-Dockerspark-Transnochados, con Docker Desktop. El servicio hive-server ejecuta HiveServer2 y yarn-resourcemanager aloja el NameNode. Las consultas se ejecutaron realmente desde PowerShell en la terminal integrada de VS Code, mediante Beeline y JDBC. Las capturas muestran el comando SQL, Get-Date del sistema anfitrión y la salida de Hive; Select-Object limita únicamente la cantidad de líneas visibles en las consultas extensas.

a. Crear una base de datos denominada **db3_john** con el comando:
create database **db3_john;**

Se creó la base db3_john y se verificó su existencia mediante SHOW DATABASES. Esta base organiza las tablas de entrada y análisis del laboratorio.



Creación y verificación de db3_john desde la terminal integrada.

b. Identifique qué columnas tiene su archivo **data3_john.csv** y determine los tipos de datos que le corresponden:

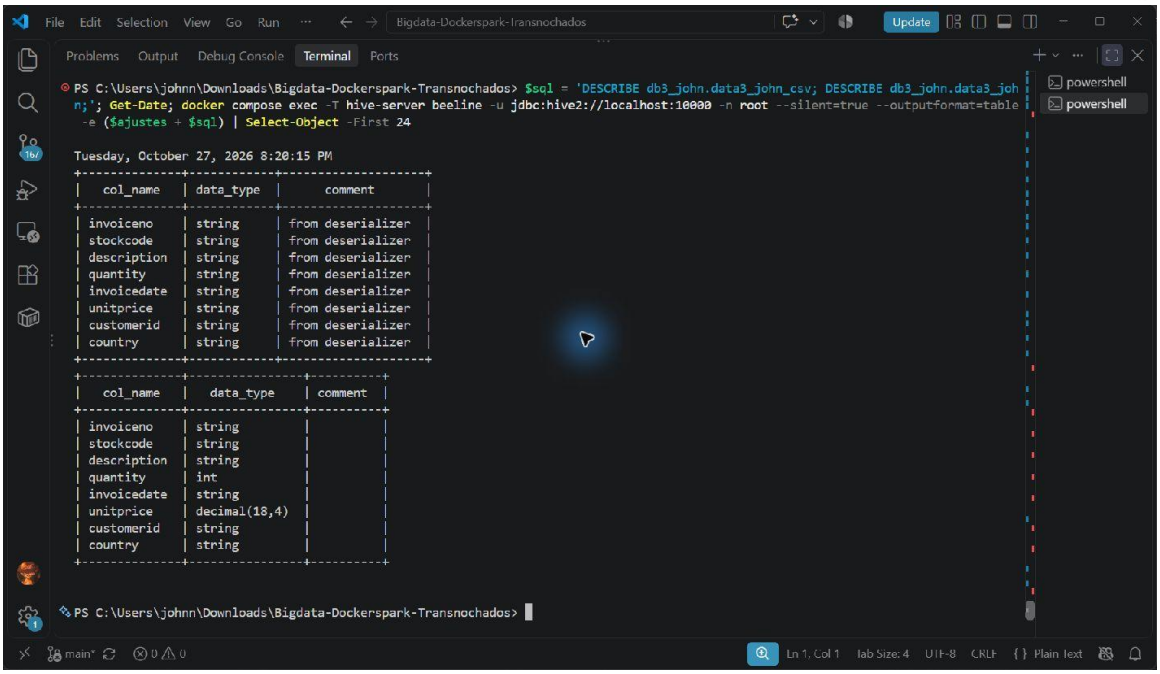
Columna	Tipo de dato	Comentario del analista	Comentario del profesor
InvoiceNo	STRING	Contiene números de factura y cancelaciones con prefijo C. INT perdería estas últimas.	Reservado para revisión del docente.
StockCode	STRING	Los códigos pueden incluir letras; existen conceptos especiales como DOT, M y AMAZONFEE.	Reservado para revisión del docente.
Description	STRING	Puede estar vacía, contener comas entre comillas o registrar ajustes de inventario. Se utiliza OpenCSVSerde.	Reservado para revisión del docente.
Quantity	INT	Admite valores positivos y negativos. Los negativos se conservan para estudiar devoluciones y ajustes.	Reservado para revisión del docente.
InvoiceDate	STRING	Se conserva el patrón M/d/yyyy H:mm requerido por los SQL.	Reservado para revisión del docente.

		Se convierte con unix_timestamp en los análisis temporales; agrupar solo por mes mezcla años.	
UnitPrice	DECIMAL(18, 4)	Representa importes monetarios con precisión decimal. Se conservan precios cero y negativos; cada consulta establece sus filtros.	Reservado para revisión del docente.
CustomerID	STRING	Identificador sin operaciones aritméticas. Hay 135.080 filas sin ID, convertidas a NULL. El texto mantiene compatibilidad con la comprobación de cadena vacía.	Reservado para revisión del docente.
Country	STRING	Permite agrupar por país. Algunas categorías representan territorios o denominaciones comerciales; no se normalizaron para conservar el dataset.	Reservado para revisión del docente.

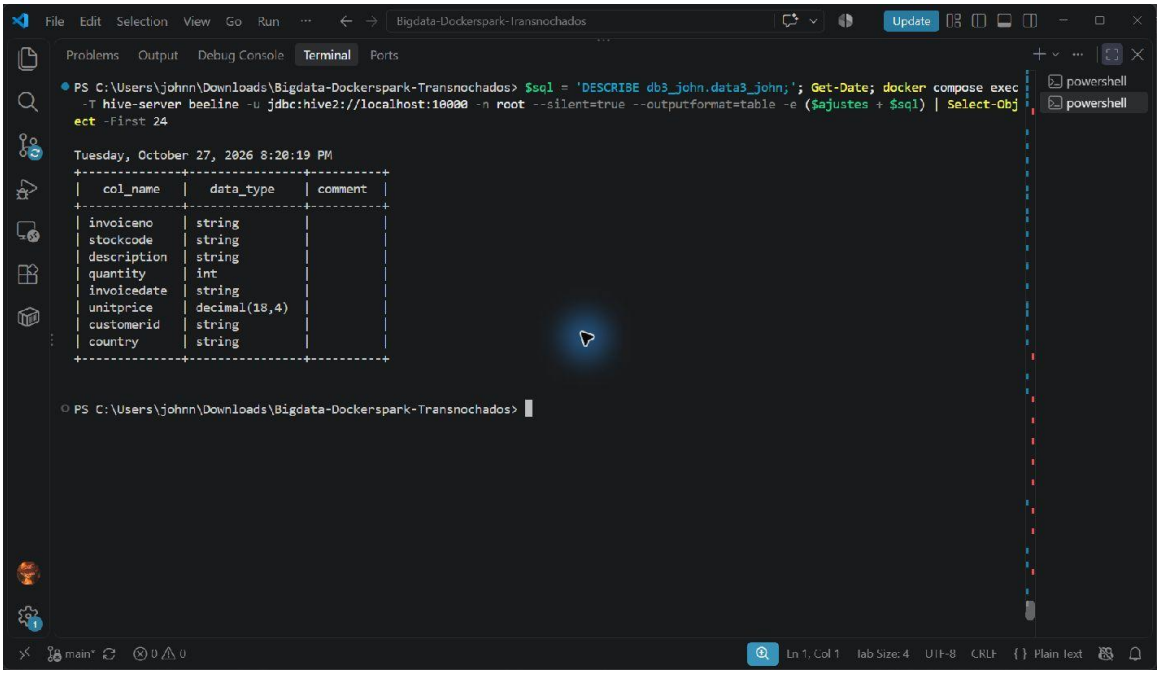
La tabla debe incluir el análisis por cada campo y declararlo en la tabla en las columnas: “columna”, “tipo de dato” y “comentario del analista” (este refiere a los campos que tengan particularidades que puedan complicar su tratamiento).

c. Crear una tabla denominada **data3_john** tomando en cuenta los tipos de datos definidos anteriormente (revisa tu anterior práctica para guiarte).

Se creó data3_john_csv con ocho columnas de texto y OpenCSVSerde para leer correctamente el CSV. A partir de ella se generó db3_john.data3_john en formato ORC, convirtiendo Quantity a INT y UnitPrice a DECIMAL(18,4). InvoiceNo y StockCode se conservaron como texto, y CustomerID vacío se convirtió a NULL. El esquema final respeta las particularidades identificadas.



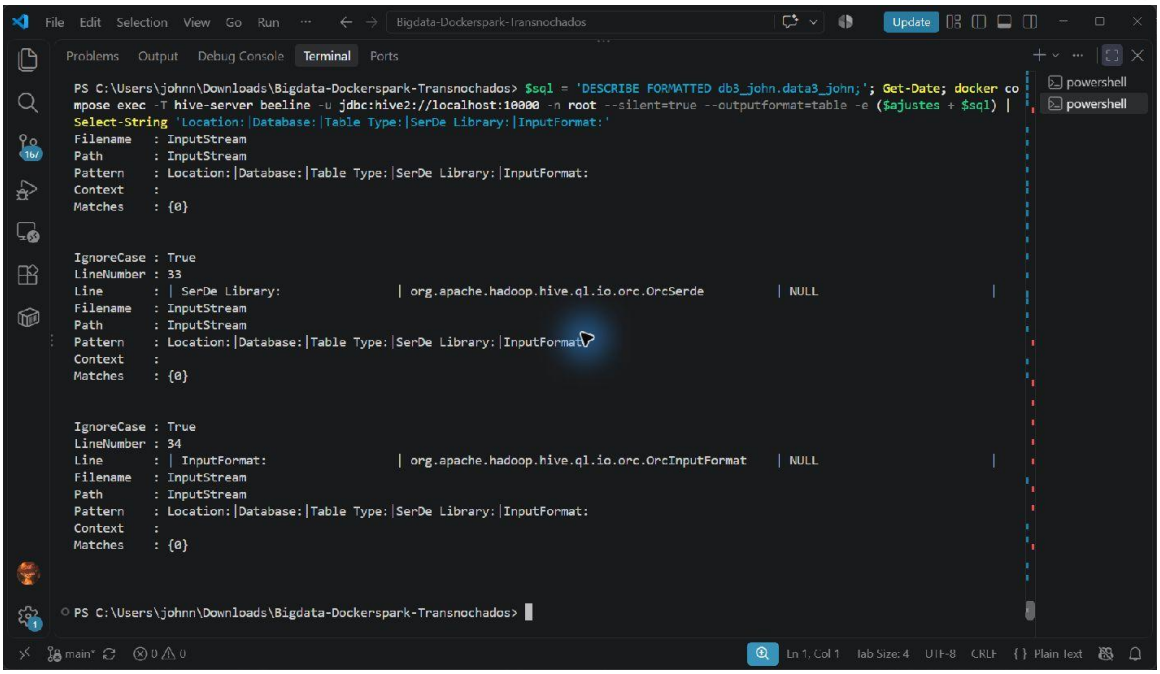
Descripción de las tablas de entrada y análisis en Hive.



Tipos de las ocho columnas de db3_john.data3_john.

d. Revisa que el valor de location llegue hasta la tabla creada **data3_john**con el comando: DESCRIBE FORMATTED **db3_john**.**data3_john**;

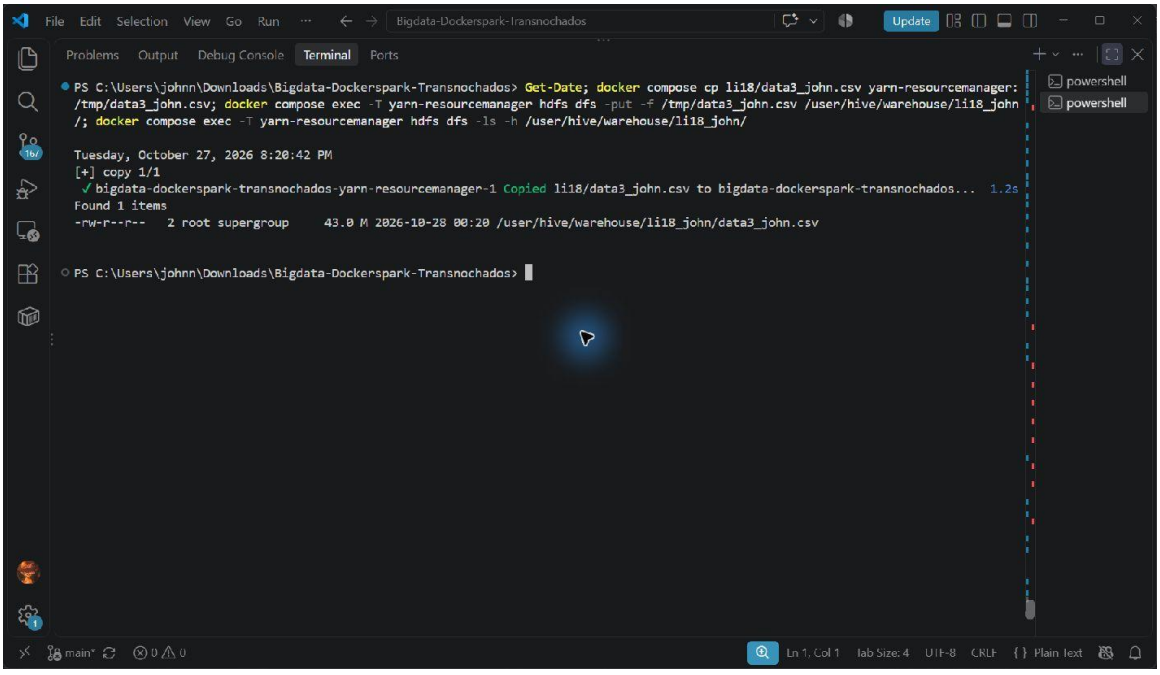
DESCRIBE FORMATTED confirmó la ubicación hdfs://yarn-resourcemanager:9000/user/hive/warehouse/db3_john.db/data3_john. La tabla es MANAGED_TABLE y utiliza OrcSerde: Hive mantiene sus metadatos y administra los archivos ORC en HDFS.



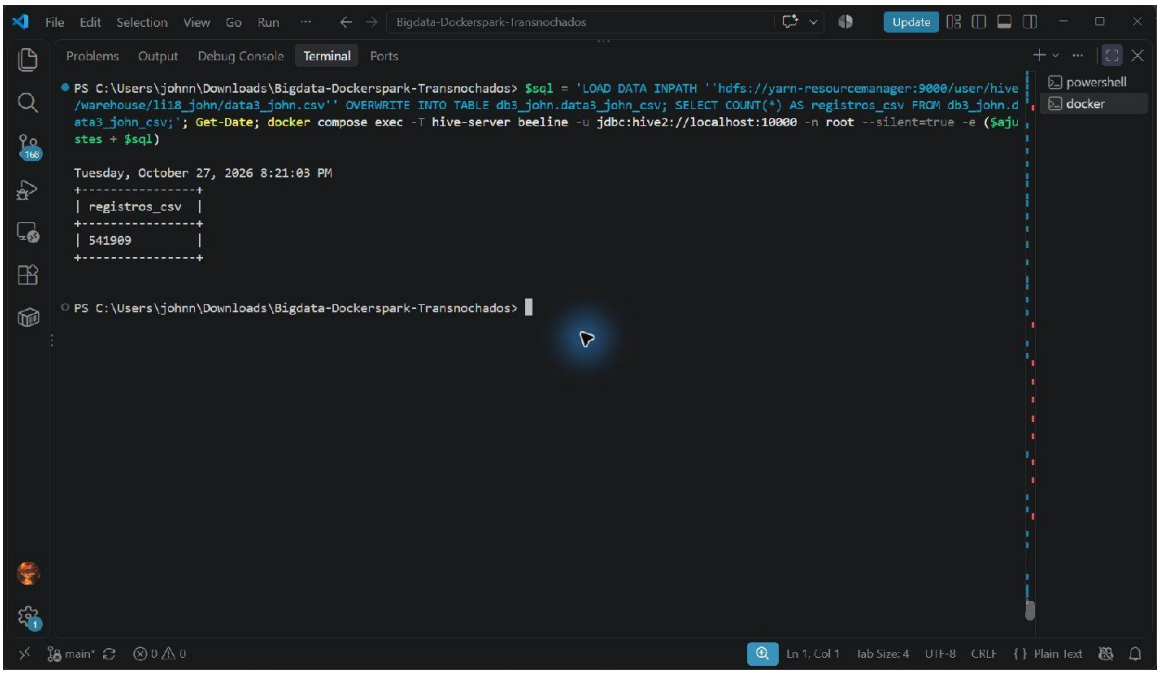
Propiedades de ubicación y almacenamiento consultadas directamente en Hive.

e. Cargar los datos del CSV a la tabla creada.

Se copió data3_john.csv a /tmp/ del contenedor y se subió a /user/hive/warehouse/li18_john/. LOAD DATA INPATH trasladó el archivo a la tabla de entrada. La evidencia repite la carga con OVERWRITE para mostrar su ejecución en pantalla sin duplicar registros. El conteo de la tabla de entrada se mantuvo en 541.909; la tabla final contiene la conversión tipada del mismo dataset.

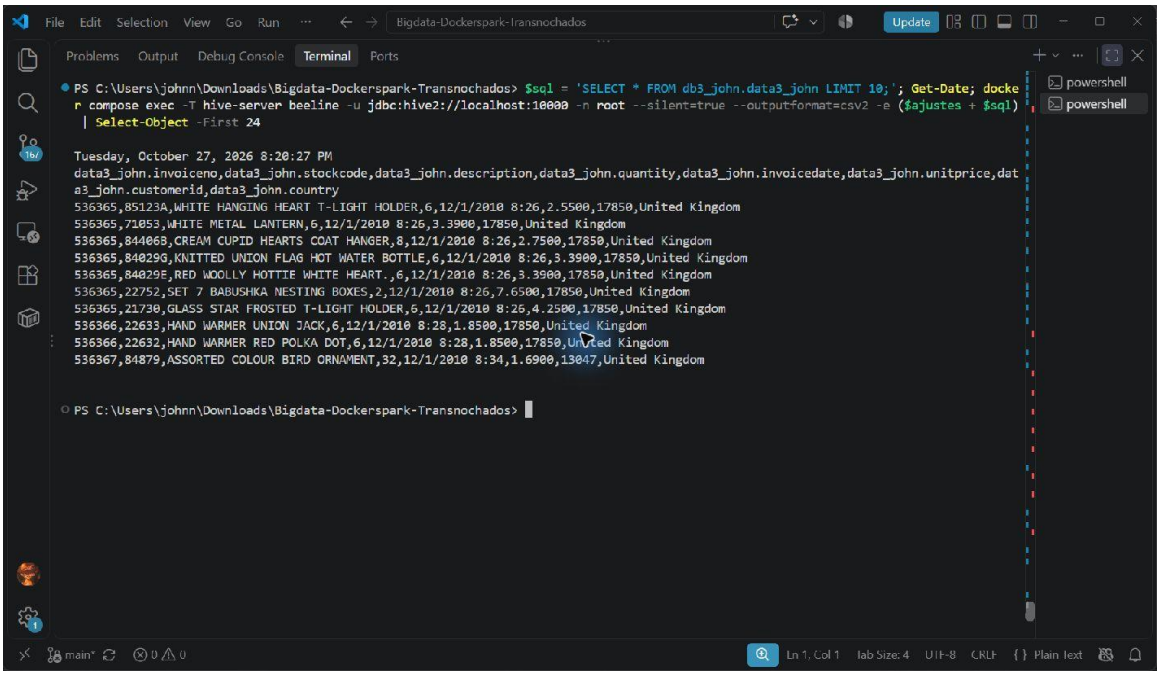


Copia al contenedor y carga del CSV en HDFS.



LOAD DATA y verificación de 541.909 registros en la tabla de entrada.

f. Mostrar los primeros 10 registros para validar que la carga se realizó correctamente. SELECT * FROM db3_john.data3_john LIMIT 10 recuperó diez registros. La salida CSV permitió visualizar las ocho columnas sin perder el contenido por el ancho de la terminal. Esta muestra valida la lectura inicial; el conteo general completa la comprobación de carga.

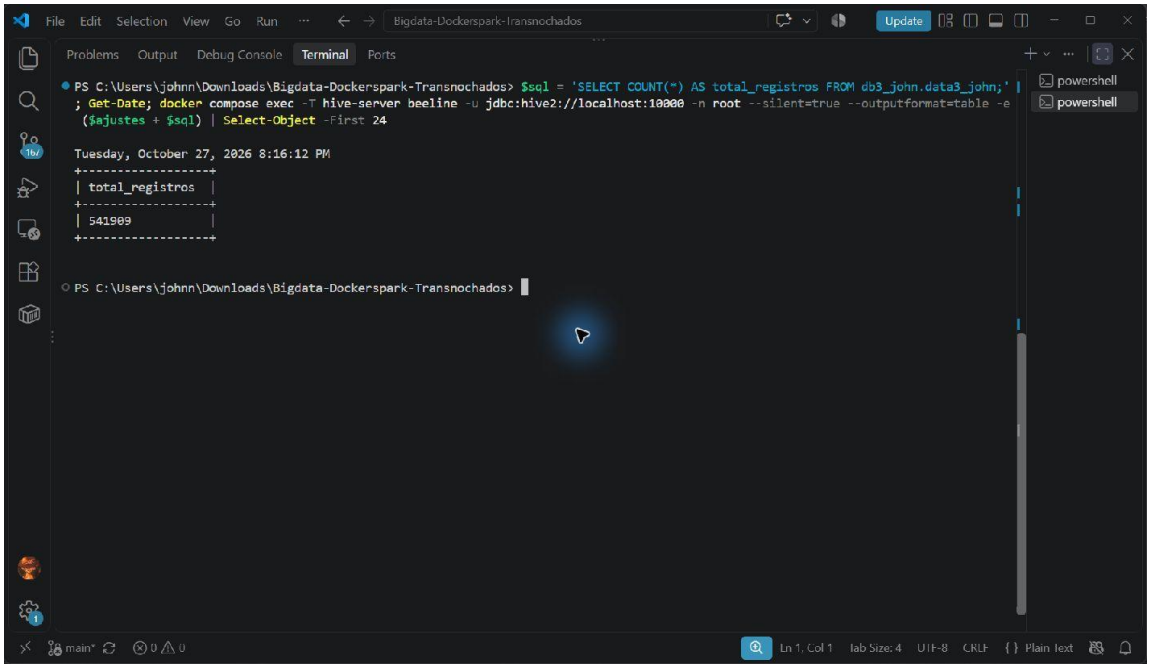


Diez registros reales consultados desde la terminal de VS Code.

4. Los siguientes son preguntas que ya vienen con su respuesta y una descripción del KPI que se está obteniendo; en este punto particular deberá:
   a. Usted debe primero comprender la pregunta.
   b. Debe probar cada SQL sugerido y considerar a qué pregunta responde el mismo.
   c. Revisar si la descripción del resultado corresponde a la pregunta.

d. Emitir la respuesta a la pregunta en sus palabras.

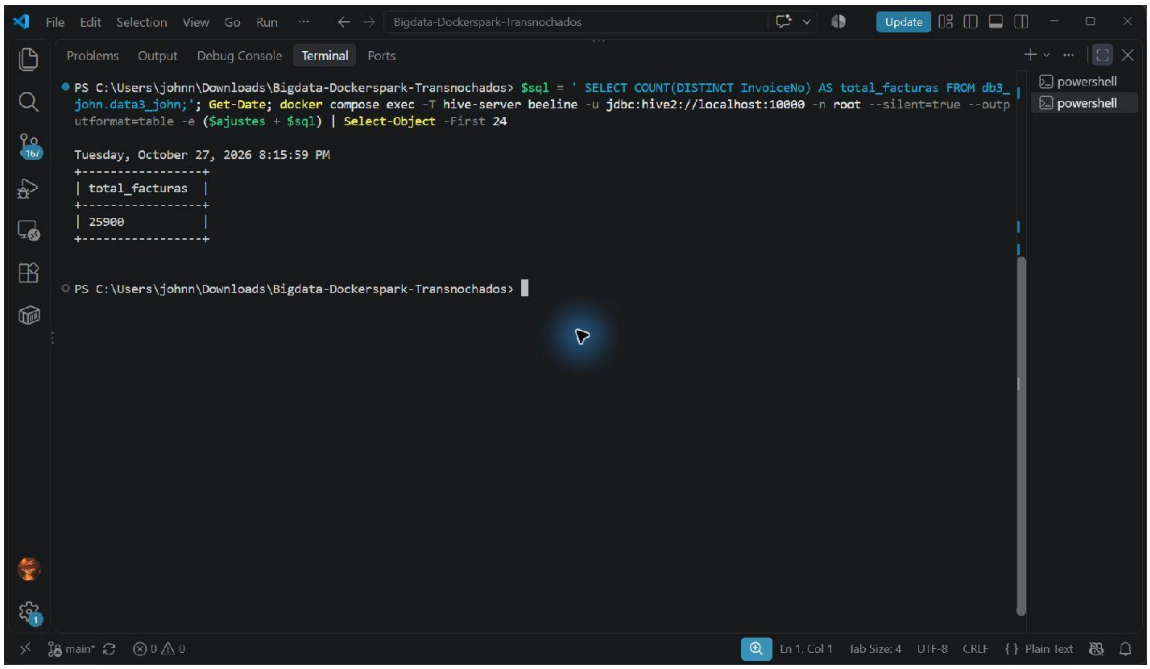
Nro.	Pregunta	SQL solución	Descripción de resultado
1	¿Cuántos registros existen en el dataset?	SELECT COUNT(*) AS total_registros FROM db3_john.data3_john;	Muestra la cantidad total de filas cargadas en Hive. Sirve para validar si el dataset fue importado correctamente.



Evidencia 1. SQL y resultado en la terminal integrada de VS Code.

Respuesta y revisión de la descripción: Se obtuvieron 541.909 registros. COUNT(*) cuenta todas las filas, incluidas devoluciones y operaciones sin cliente identificado. La descripción corresponde al resultado y el conteo coincide con los registros convertidos desde Excel.

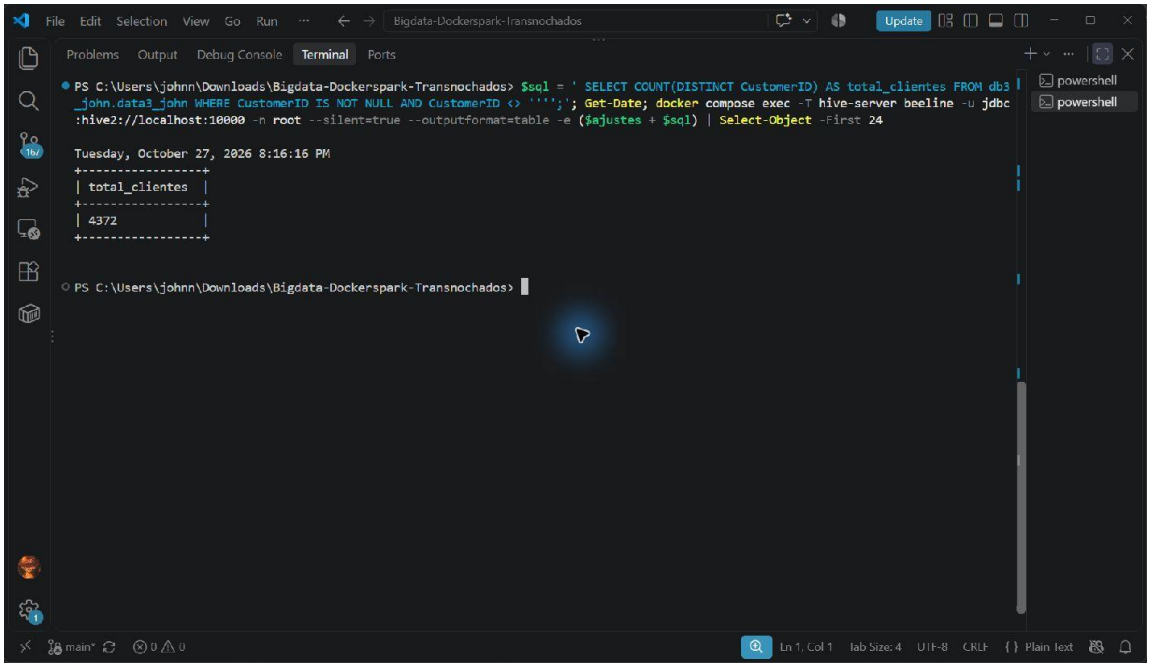
2	¿Cuántas facturas únicas existen?	SELECT COUNT(DISTINCT InvoiceNo) AS total_facturas FROM db3_john.data3_john;	Indica cuántas facturas o transacciones únicas existen en el dataset. Una factura puede tener varios productos asociados.



Evidencia 2. SQL y resultado en la terminal integrada de VS Code.

Respuesta y revisión de la descripción: Se identificaron 25.900 valores distintos de InvoiceNo. La consulta cuenta facturas y documentos con prefijo C, no solo compras positivas. Es correcta como inventario de identificadores de factura, pero el resultado no equivale a 25.900 ventas válidas.

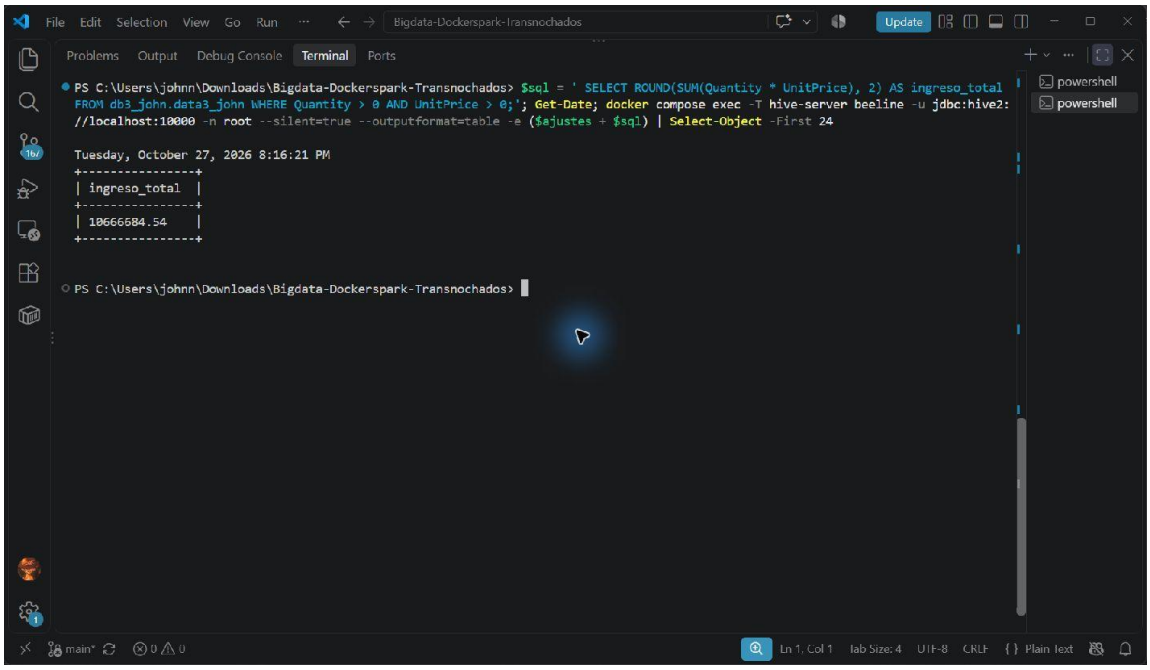
3	¿Cuántos clientes únicos realizaron compras?	SELECT COUNT(DISTINCT CustomerID) AS total_clientes FROM db3_john.data3_john WHERE CustomerID IS NOT NULL AND CustomerID <> '';	Muestra el número de clientes distintos registrados en las ventas. Se excluyen clientes nulos o vacíos.



Evidencia 3. SQL y resultado en la terminal integrada de VS Code.

Respuesta y revisión de la descripción: Se obtuvieron 4.372 clientes identificados distintos. El SQL excluye identificadores nulos o vacíos, pero no exige cantidades ni precios positivos. Por ello responde al número de clientes presentes en el dataset, incluidos quienes figuran en devoluciones, y no estrictamente a clientes con compras válidas.

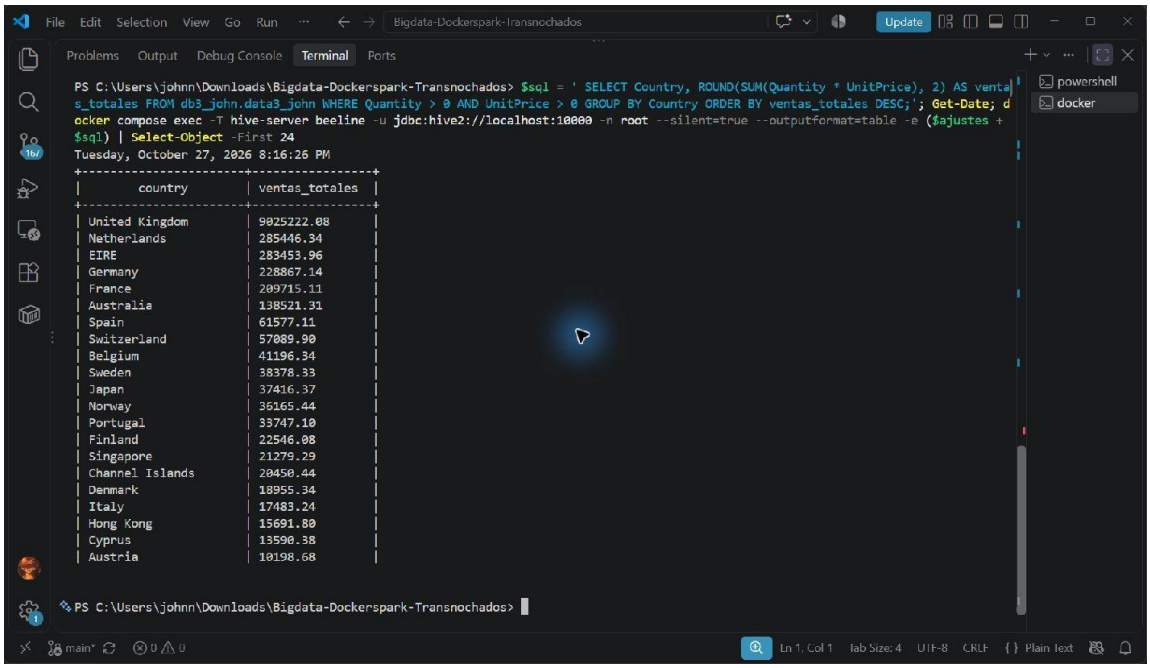
4	¿Cuál es el ingreso total generado por ventas válidas?	SELECT ROUND(SUM(Quantity * UnitPrice), 2) AS ingreso_total FROM db3_john.data3_john WHERE Quantity > 0 AND UnitPrice > 0;	Calcula el ingreso total considerando solo ventas positivas. No toma en cuenta devoluciones ni registros con precio cero.



Evidencia 4. SQL y resultado en la terminal integrada de VS Code.

Respuesta y revisión de la descripción: El ingreso total de las líneas con Quantity > 0 y UnitPrice > 0 fue de 10.666.684,54. Se sumó cantidad por precio unitario. La consulta incluye clientes anónimos y corresponde a ventas brutas positivas; no representa utilidad ni ingreso neto después de devoluciones.

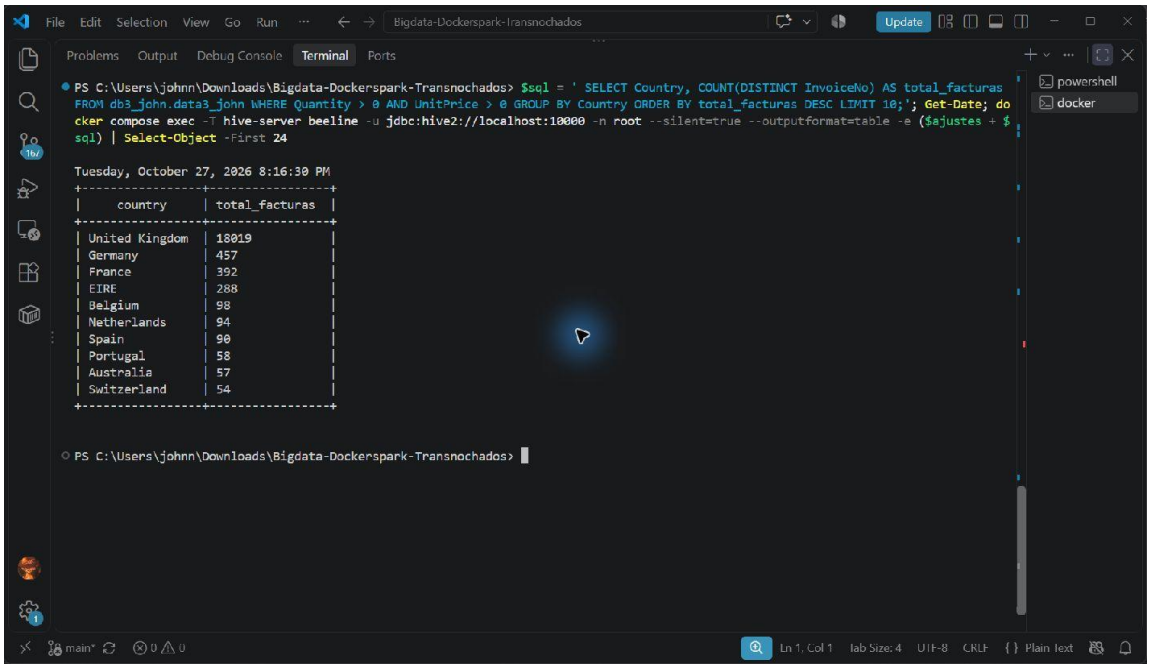
5	¿Cuál es el total de ventas por país?	SELECT Country, ROUND(SUM(Quantity * UnitPrice), 2) AS ventas_totales FROM db3_john.data3_john WHERE Quantity > 0 AND UnitPrice > 0 GROUP BY Country ORDER BY ventas_totales DESC;	Muestra qué países generan mayor volumen de ventas. Permite identificar los mercados más importantes para la empresa.



Evidencia 5. SQL y resultado en la terminal integrada de VS Code.

Respuesta y revisión de la descripción: United Kingdom encabezó las ventas positivas con 9.025.222,08, seguido de Netherlands con 285.446,34 y EIRE con 283.453,96. Se obtuvieron 38 categorías de país. El agrupamiento responde al importe por mercado, no a unidades vendidas, y no descuenta devoluciones.

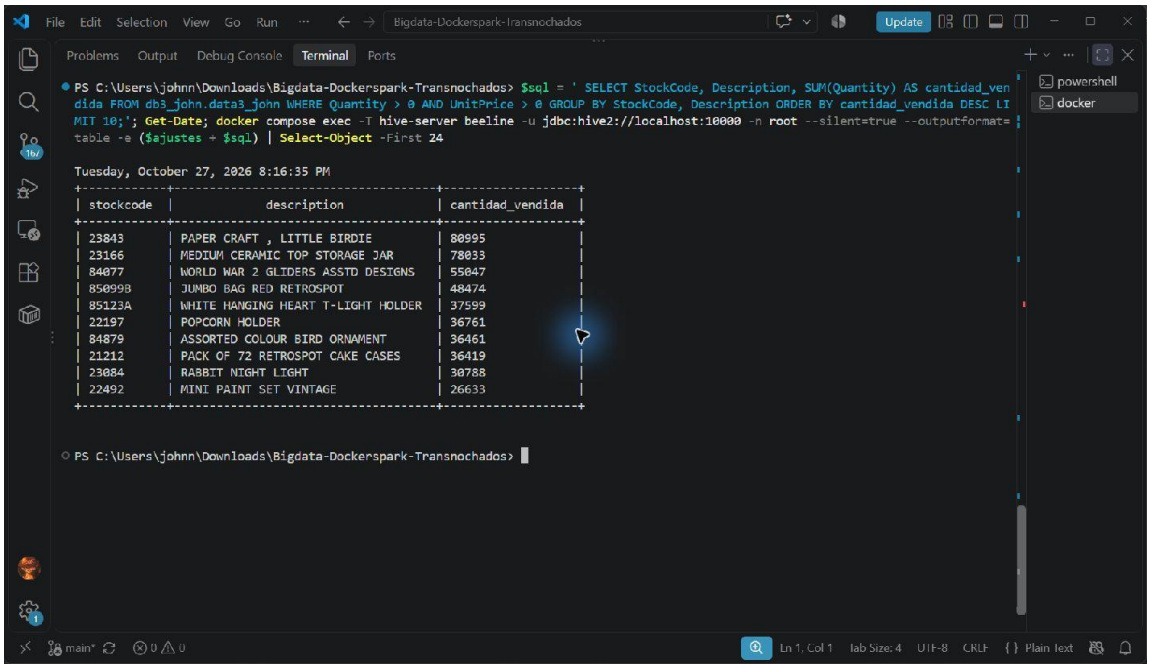
6	¿Cuáles son los 10 países con mayor cantidad de facturas?	SELECT Country, COUNT(DISTINCT InvoiceNo) AS total_facturas FROM db3_john.data3_john WHERE Quantity > 0 AND UnitPrice > 0 GROUP BY Country ORDER BY total_facturas DESC LIMIT 10;	Muestra los países con mayor número de transacciones. Sirve para analizar frecuencia comercial por mercado.



Evidencia 6. SQL y resultado en la terminal integrada de VS Code.

Respuesta y revisión de la descripción: United Kingdom registró 18.019 facturas con líneas de cantidad y precio positivos; Germany, 457; y France, 392. Se mostraron los diez primeros países. La consulta compara frecuencia mediante documentos distintos por país y no cantidad de filas o productos.

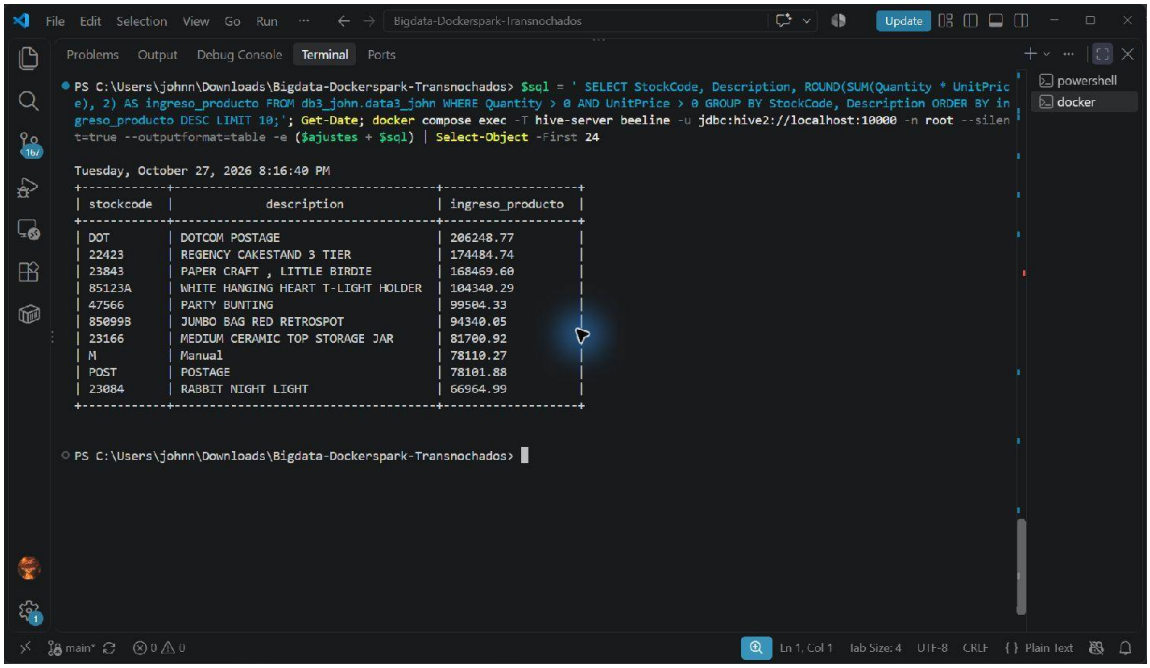
7	¿Cuáles son los 10 productos más vendidos por cantidad?	SELECT StockCode, Description, SUM(Quantity) AS cantidad_vendida FROM db3_john.data3_john WHERE Quantity > 0 AND UnitPrice > 0 GROUP BY StockCode, Description ORDER BY cantidad_vendida DESC LIMIT 10;	Identifica los productos con mayor volumen vendido en unidades. Es útil para decisiones de inventario y abastecimiento.



Evidencia 7. SQL y resultado en la terminal integrada de VS Code.

Respuesta y revisión de la descripción: El código 23843, PAPER CRAFT, LITTLE BIRDIE, fue primero con 80.995 unidades; el 23166, MEDIUM CERAMIC TOP STORAGE JAR, registró 78.033; y el 84077, WORLD WAR 2 GLIDERS ASSTD DESIGNS, 55.047. El SQL suma cantidades positivas por código y descripción. Responde al volumen bruto vendido, sin descontar devoluciones; el mismo código con descripciones distintas se separa.

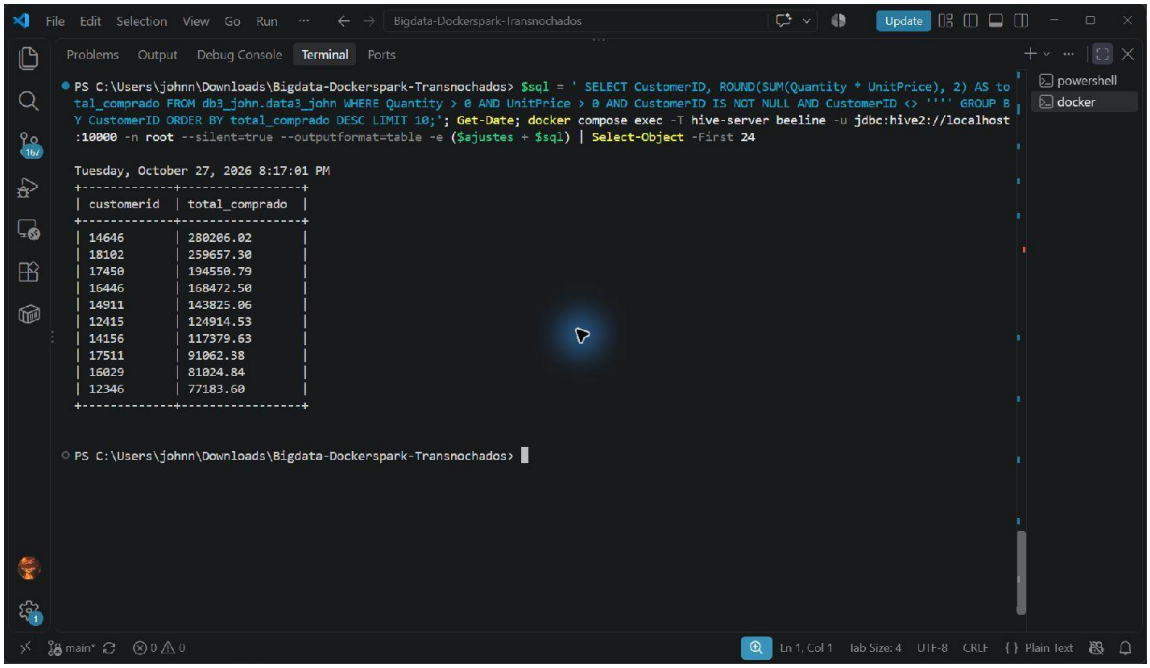
8	¿Cuáles son los 10 productos que generan más ingresos?	SELECT StockCode, Description, ROUND(SUM(Quantity * UnitPrice), 2) AS ingreso_producto FROM db3_john.data3_john WHERE Quantity > 0 AND UnitPrice > 0 GROUP BY StockCode, Description ORDER BY ingreso_producto DESC LIMIT 10;	Muestra los productos que más dinero generan, aunque no necesariamente sean los más vendidos en cantidad.



Evidencia 8. SQL y resultado en la terminal integrada de VS Code.

Respuesta y revisión de la descripción: DOT, DOTCOM POSTAGE, generó 206.248,77; 22423, REGENCY CAKESTAND 3 TIER, 174.484,74; y 23843, PAPER CRAFT, LITTLE BIRDIE, 168.469,60. El SQL ordena el importe positivo por código y descripción. La descripción del indicador es adecuada, pero DOTCOM POSTAGE es un concepto de envío y no necesariamente un producto físico.

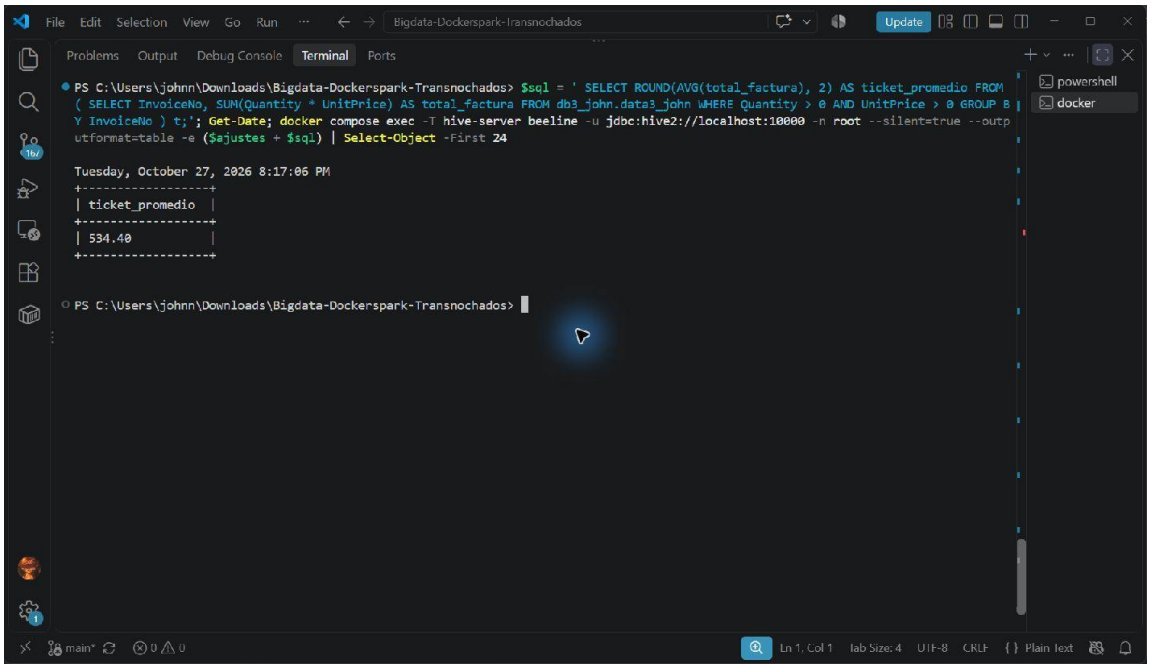
9	¿Cuáles son los clientes que más compraron en valor monetario?	SELECT CustomerID, ROUND(SUM(Quantity * UnitPrice), 2) AS total_comprado FROM db3_john.data3_john WHERE Quantity > 0 AND UnitPrice > 0 AND CustomerID IS NOT NULL AND CustomerID <> '' GROUP BY CustomerID ORDER BY total_comprado DESC LIMIT 10;	Identifica los clientes más valiosos según el monto total comprado. Sirve para segmentación comercial y fidelización.



Evidencia 9. SQL y resultado en la terminal integrada de VS Code.

Respuesta y revisión de la descripción: El cliente 14646 lideró con 280.206,02; le siguieron 18102 con 259.657,30 y 17450 con 194.550,79. Se sumaron ventas positivas de clientes identificados. La consulta responde al mayor valor bruto comprado, no a rentabilidad ni gasto neto después de devoluciones.

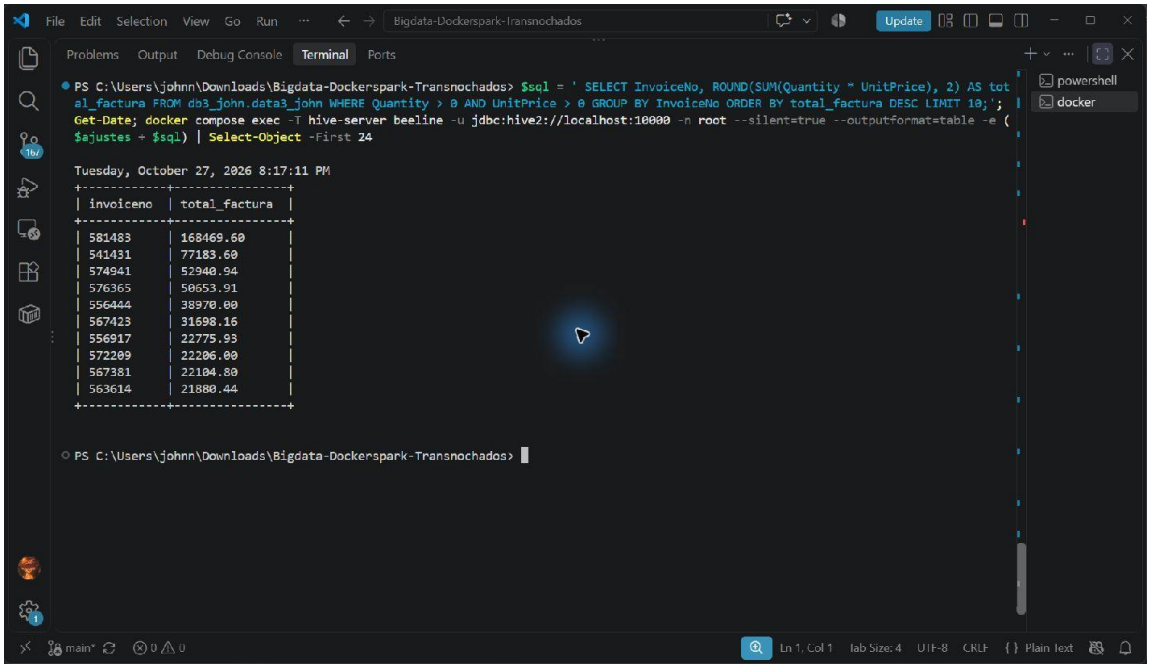
10	¿Cuál es el ticket promedio por factura?	SELECT ROUND(AVG(total_factura), 2) AS ticket_promedio FROM (SELECT InvoiceNo, SUM(Quantity * UnitPrice) AS total_factura FROM db3_john.data3_john WHERE Quantity > 0 AND UnitPrice > 0 GROUP BY InvoiceNo) t;	Calcula cuánto gasta en promedio un cliente por factura. Es un indicador clave para evaluar comportamiento de compra.



Evidencia 10. SQL y resultado en la terminal integrada de VS Code.

Respuesta y revisión de la descripción: El ticket promedio por factura fue de 534,40. La subconsulta suma las líneas positivas de cada InvoiceNo y la consulta exterior promedia esos totales. Corresponde al importe medio por pedido, no al promedio de líneas ni al gasto total de cada cliente.

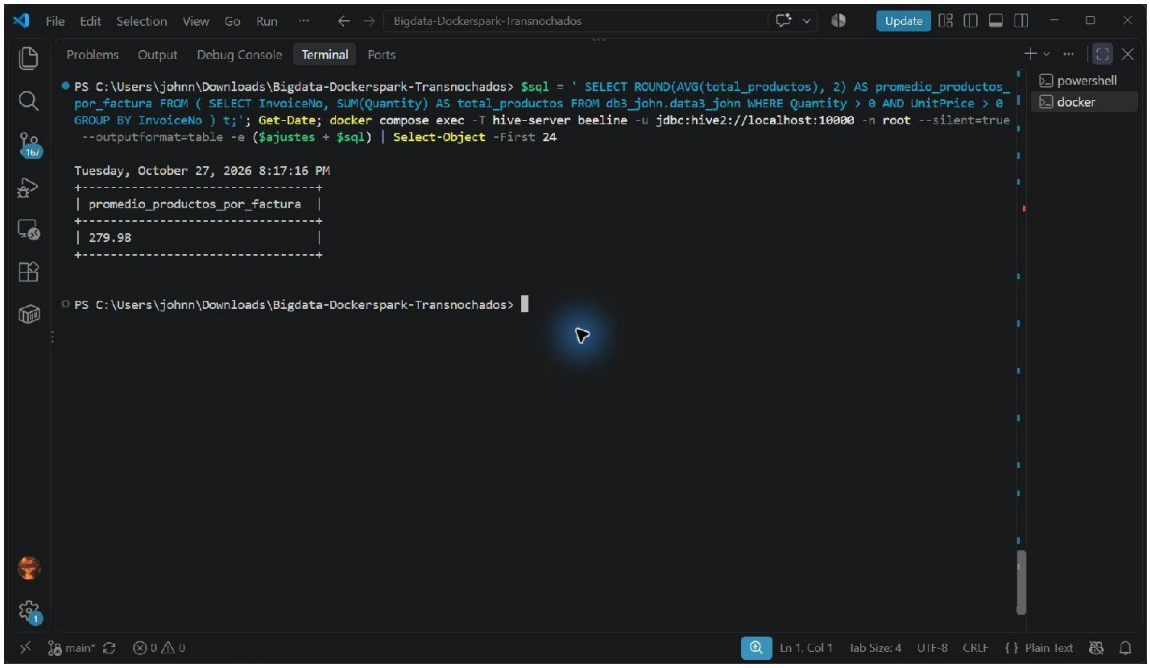
11	¿Cuáles son las facturas con mayor monto total?	SELECT InvoiceNo, ROUND(SUM(Quantity * UnitPrice), 2) AS total_factura FROM db3_john.data3_john WHERE Quantity > 0 AND UnitPrice > 0 GROUP BY InvoiceNo ORDER BY total_factura DESC LIMIT 10;	Muestra las facturas de mayor valor económico. Ayuda a detectar grandes compras o pedidos mayoristas.



Evidencia 11. SQL y resultado en la terminal integrada de VS Code.

Respuesta y revisión de la descripción: La factura 581483 fue la de mayor importe, con 168.469,60, seguida de 541431 con 77.183,60 y 574941 con 52.940,94. La suma por documento permite identificar órdenes grandes. Sus importes se calculan con líneas positivas y no garantizan que las compras no hayan sido canceladas posteriormente.

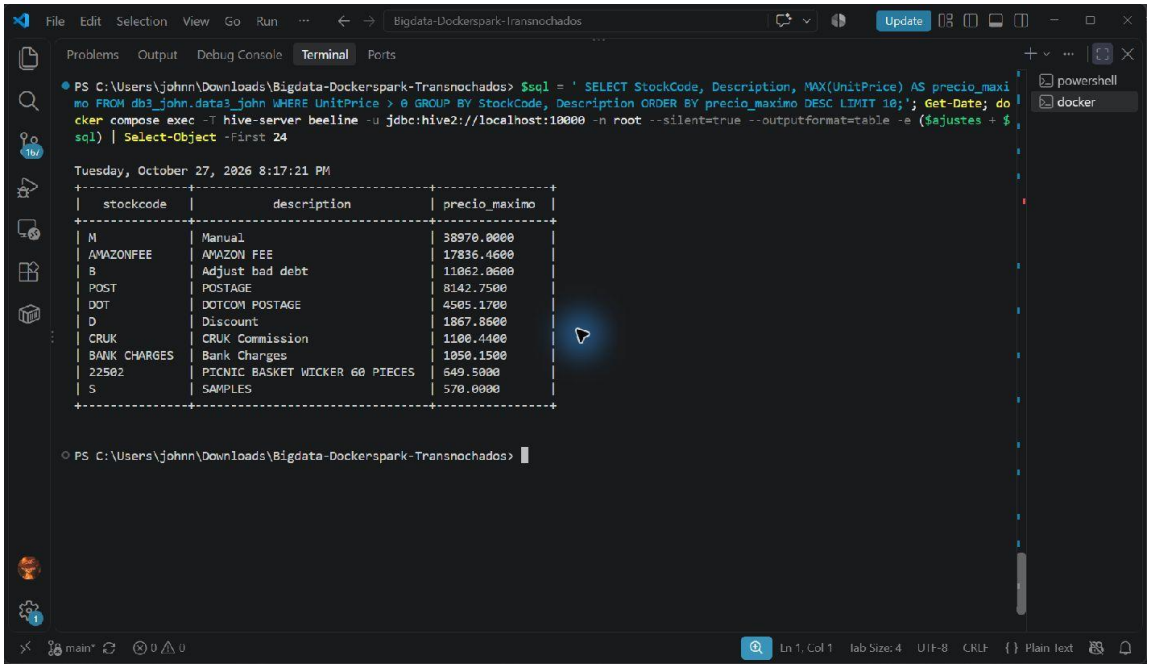
12	¿Cuál es la cantidad promedio de productos por factura?	SELECT ROUND(AVG(total_productos), 2) AS promedio_productos_por_factura FROM (SELECT InvoiceNo, SUM(Quantity) AS total_productos FROM db3_john.data3_john WHERE Quantity > 0 AND UnitPrice > 0 GROUP BY InvoiceNo) t;	Calcula cuántas unidades se compran en promedio por factura. Sirve para analizar el tamaño promedio de los pedidos.



Evidencia 12. SQL y resultado en la terminal integrada de VS Code.

Respuesta y revisión de la descripción: El promedio fue de 279,98 unidades por factura. Se sumó Quantity por documento y se promediaron esos totales. La consulta responde a unidades, no a cantidad de productos diferentes ni a número de líneas: un mismo artículo puede aportar muchas unidades.

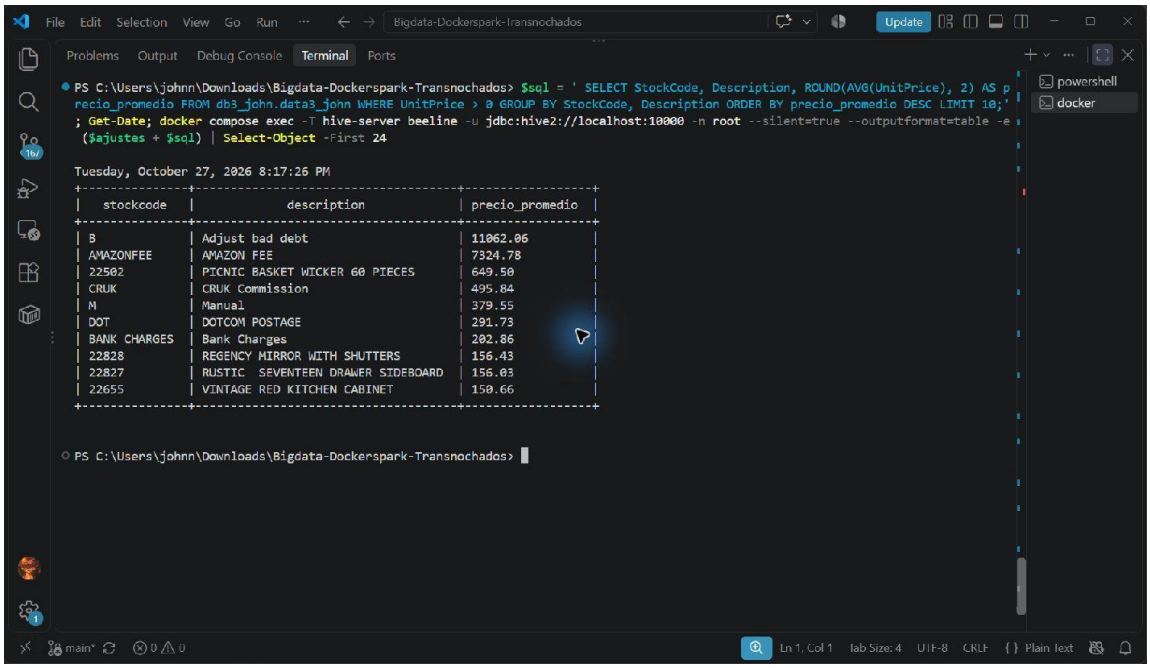
13	¿Cuáles son los productos con precio unitario más alto?	SELECT StockCode, Description, MAX(UnitPrice) AS precio_maximo FROM db3_john.data3_john WHERE UnitPrice > 0 GROUP BY StockCode, Description ORDER BY precio_maximo DESC LIMIT 10;	Identifica los productos con mayor precio unitario registrado. Puede servir para analizar productos premium o registros atípicos.



Evidencia 13. SQL y resultado en la terminal integrada de VS Code.

Respuesta y revisión de la descripción: El mayor precio registrado fue 38.970,0000 para M, Manual; AMAZONFEE alcanzó 17.836,4600 y B, Adjust bad debt, 11.062,0600. El SQL busca el máximo precio positivo por código y descripción sin filtrar cantidades negativas. Identifica valores altos y atípicos, pero varios son cargos o ajustes, por lo que no corresponde tratarlos automáticamente como productos premium.

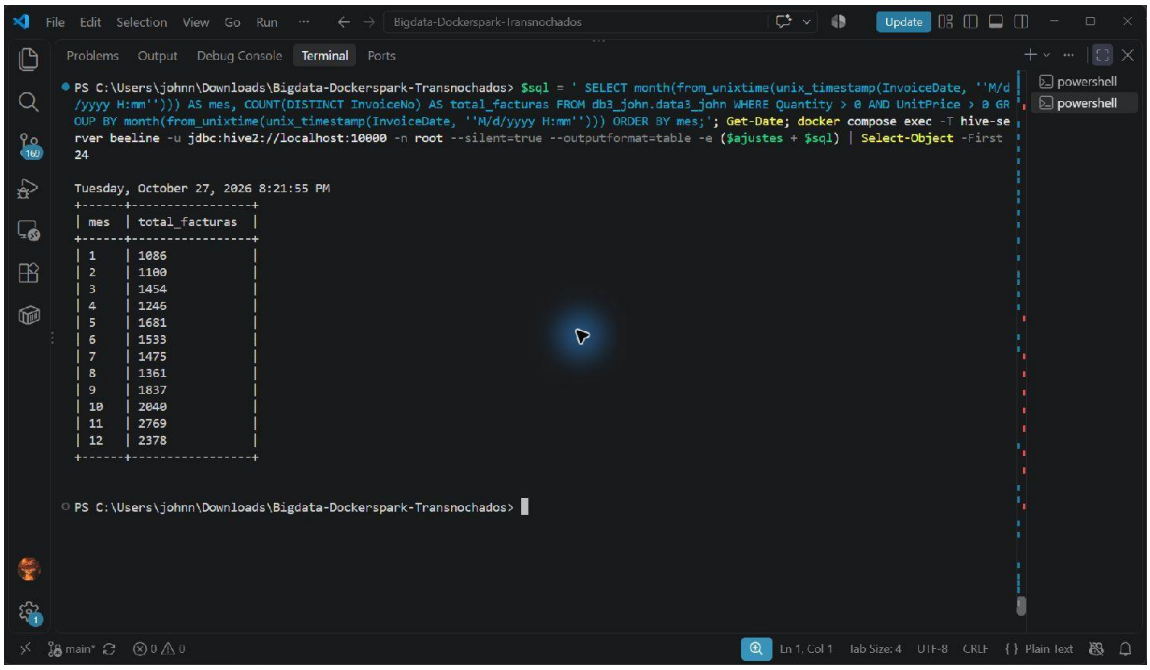
14	¿Cuáles son los productos con precio unitario promedio más alto?	SELECT StockCode, Description, ROUND(AVG(UnitPrice), 2) AS precio_promedio FROM db3_john.data3_john WHERE UnitPrice > 0 GROUP BY StockCode, Description ORDER BY precio_promedio DESC LIMIT 10;	Muestra los productos con mayor precio promedio. Es útil cuando un producto aparece varias veces con precios distintos.



Evidencia 14. SQL y resultado en la terminal integrada de VS Code.

Respuesta y revisión de la descripción: B, Adjust bad debt, tuvo el mayor precio promedio: 11.062,06. AMAZONFEE promedió 7.324,78 y 22502, PICNIC BASKET WICKER 60 PIECES, 649,50. AVG calcula un promedio simple de precios por línea y no uno ponderado por unidades. Se incluyen devoluciones si el precio es positivo; el resultado requiere distinguir artículos de cargos y ajustes.

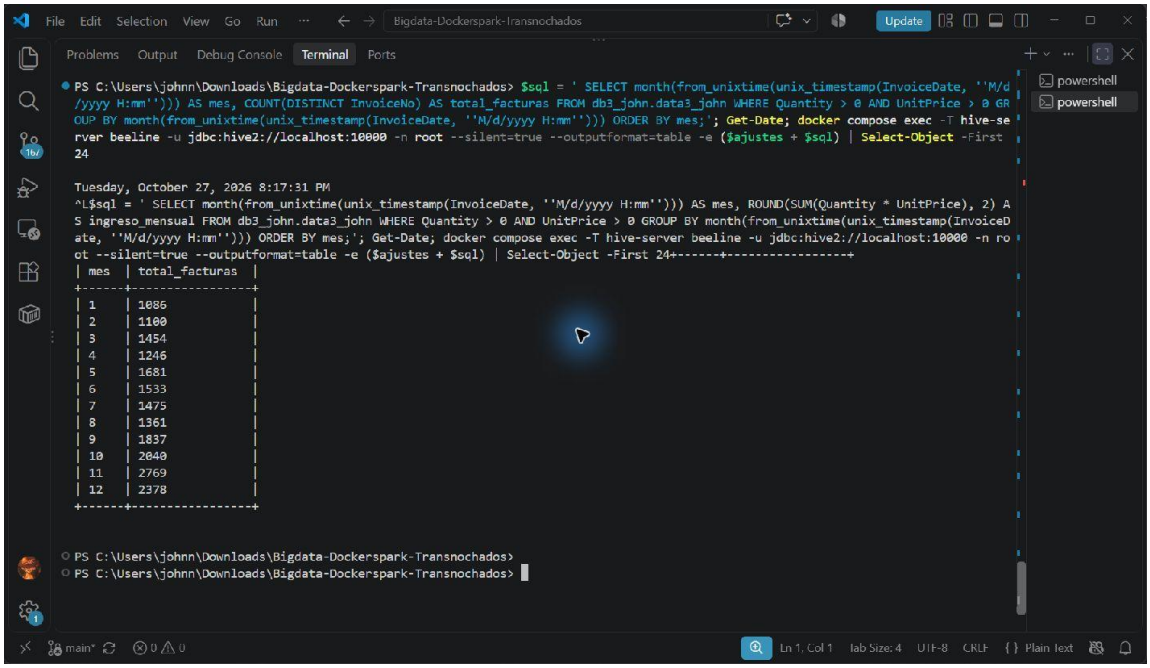
15	¿Cuántas ventas se realizaron por mes?	SELECT month(from_unixtime(unix_timestamp(InvoiceDate, 'M/d/yyyy H:mm'))) AS mes, COUNT(DISTINCT InvoiceNo) AS total_facturas FROM db3_john.data3_john WHERE Quantity > 0 AND UnitPrice > 0 GROUP BY month(from_unixtime(unix_timestamp(InvoiceDate, 'M/d/yyyy H:mm'))) ORDER BY mes;	Agrupa las ventas por mes. Permite analizar estacionalidad y detectar meses con mayor movimiento comercial.



Evidencia 15. SQL y resultado en la terminal integrada de VS Code.

Respuesta y revisión de la descripción: Noviembre concentró 2.769 facturas positivas y diciembre 2.378; enero tuvo 1.086. La conversión de fecha permitió agrupar por número de mes. La consulta responde al conteo mensual, pero combina diciembre de 2010 y diciembre de 2011 y no prueba por sí sola una estacionalidad recurrente.

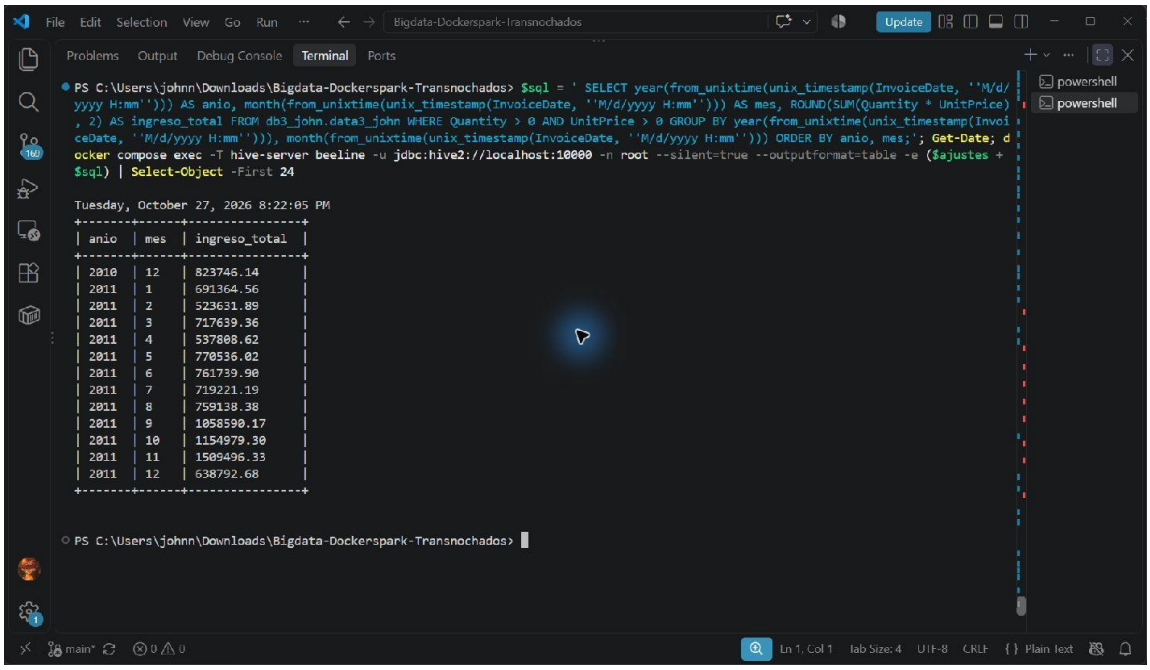
16	¿Cuál es el ingreso total por mes?	SELECT month(from_unixtime(unix_timestamp(InvoiceDate, 'M/d/yyyy H:mm'))) AS mes, ROUND(SUM(Quantity * UnitPrice), 2) AS ingreso_mensual FROM db3_john.data3_john WHERE Quantity > 0 AND UnitPrice > 0 GROUP BY month(from_unixtime(unix_timestamp(InvoiceDate, 'M/d/yyyy H:mm'))) ORDER BY mes;	Muestra el ingreso generado en cada mes. Sirve para analizar crecimiento, caída de ventas o temporadas fuertes.



Evidencia 16. SQL y resultado en la terminal integrada de VS Code.

Respuesta y revisión de la descripción: Noviembre presentó el mayor ingreso agrupado por mes, 1.509.496,33; diciembre sumó 1.462.538,82 y octubre 1.154.979,30. El SQL suma importes positivos y mezcla el mismo mes de años diferentes. Por ello describe la distribución por mes del año, no una serie cronológica completa.

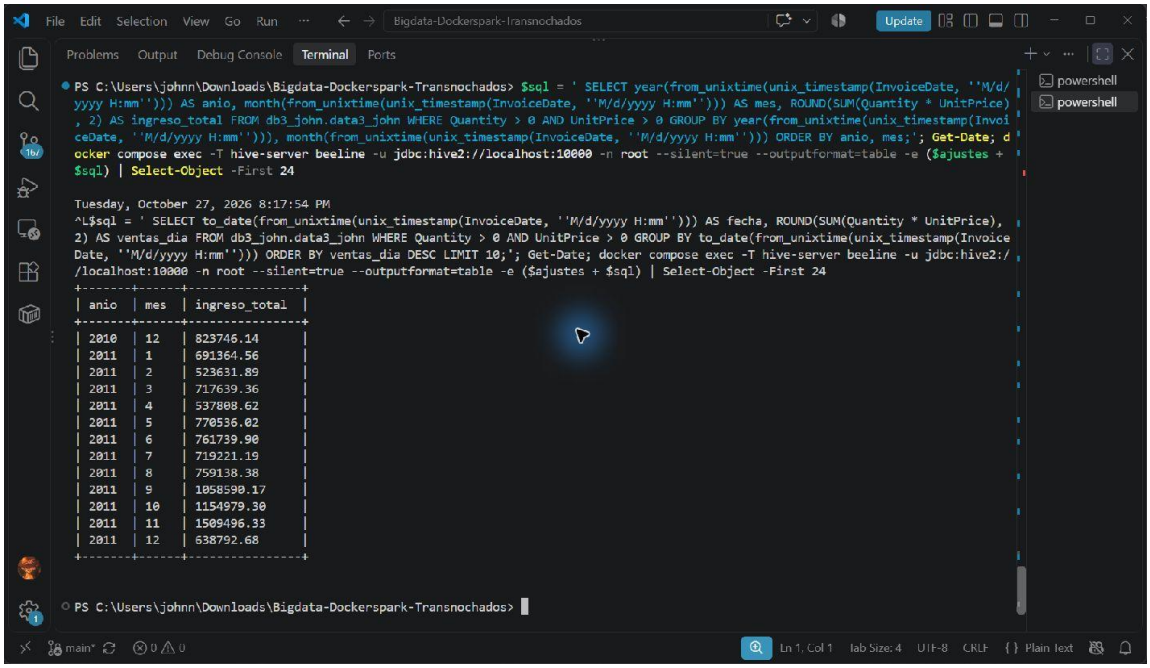
17	¿Cuál es el ingreso total por año y mes?	SELECT year(from_unixtime(unix_timestamp(InvoiceDate, 'M/d/yyyy H:mm'))) AS anio, month(from_unixtime(unix_timestamp(InvoiceDate, 'M/d/yyyy H:mm'))) AS mes, ROUND(SUM(Quantity * UnitPrice), 2) AS ingreso_total FROM db3_john.data3_john WHERE Quantity > 0 AND UnitPrice > 0 GROUP BY year(from_unixtime(unix_timestamp(InvoiceDate, 'M/d/yyyy H:mm'))), month(from_unixtime(unix_timestamp(InvoiceDate, 'M/d/yyyy H:mm'))) ORDER BY anio, mes;	Muestra las ventas agrupadas por año y mes. Es más preciso que agrupar solo por mes porque separa periodos de diferentes años.



Evidencia 17. SQL y resultado en la terminal integrada de VS Code.

Respuesta y revisión de la descripción: Al separar año y mes, noviembre de 2011 fue el período de mayor ingreso con 1.509.496,33. Diciembre de 2010 registró 823.746,14 y diciembre de 2011, 638.792,68. Esta consulta responde mejor a la evolución temporal. El último mes es incompleto, porque el dataset llega hasta el 9 de diciembre de 2011.

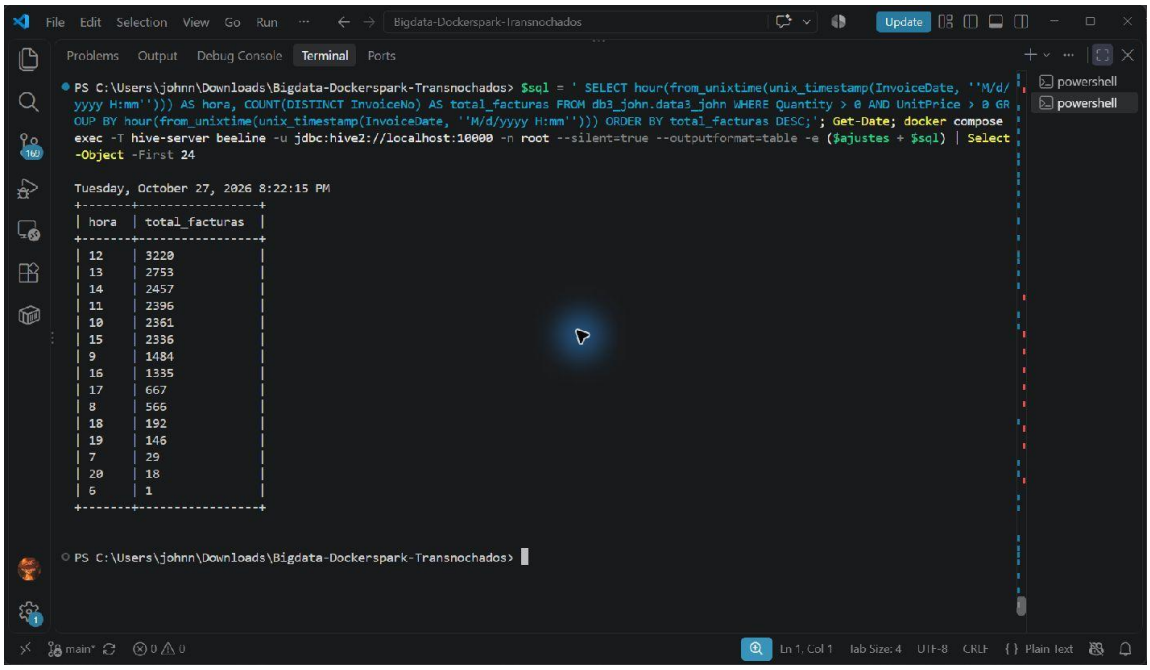
18	¿Qué día tuvo mayores ventas?	SELECT to_date(from_unixtime(unix_timestamp(InvoiceDate, 'M/d/yyyy H:mm'))) AS fecha, ROUND(SUM(Quantity * UnitPrice), 2) AS ventas_dia FROM db3_john.data3_john WHERE Quantity > 0 AND UnitPrice > 0 GROUP BY to_date(from_unixtime(unix_timestamp(InvoiceDate, 'M/d/yyyy H:mm'))) ORDER BY ventas_dia DESC LIMIT 10;	Identifica los días con mayor ingreso. Puede ayudar a detectar campañas, promociones o fechas de alta demanda.



Evidencia 18. SQL y resultado en la terminal integrada de VS Code.

Respuesta y revisión de la descripción: El día de mayor ingreso positivo fue el 9 de diciembre de 2011, con 200.920,60. Le siguieron el 14 de noviembre con 114.419,89 y el 20 de septiembre con 109.612,03. Se agrupa por fecha y se ordena la suma de ventas; el resultado permite localizar picos, pero no demuestra que fueran causados por campañas.

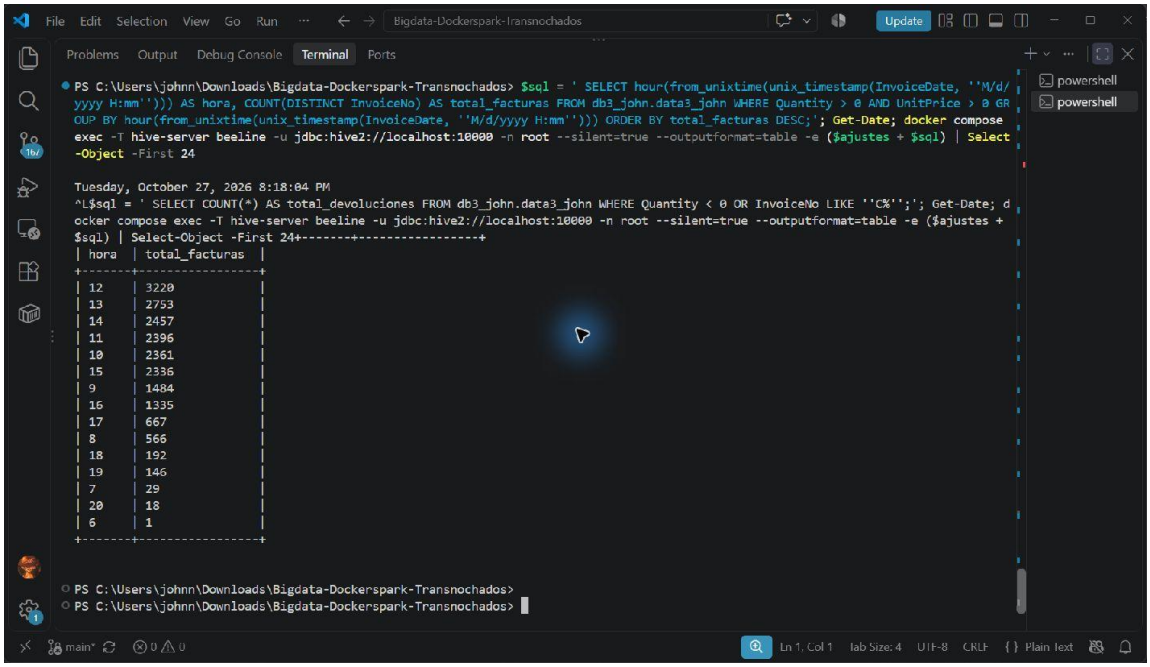
19	¿En qué horas del día se realizan más compras?	SELECT hour(from_unixtime(unix_timestamp(InvoiceDate, 'M/d/yyyy H:mm'))) AS hora, COUNT(DISTINCT InvoiceNo) AS total_facturas FROM db3_john.data3_john WHERE Quantity > 0 AND UnitPrice > 0 GROUP BY hour(from_unixtime(unix_timestamp(InvoiceDate, 'M/d/yyyy H:mm'))) ORDER BY total_facturas DESC;	Muestra las horas con mayor cantidad de facturas. Sirve para analizar patrones horarios de compra.



Evidencia 19. SQL y resultado en la terminal integrada de VS Code.

Respuesta y revisión de la descripción: Las 12:00 reunieron la mayor cantidad de facturas positivas, 3.220; las 13:00 registraron 2.753 y las 14:00, 2.457. El SQL cuenta documentos distintos por hora y responde a frecuencia de compras, no al mayor ingreso o ticket. Las horas corresponden a las fechas registradas en el dataset, sin una conversión de zona horaria.

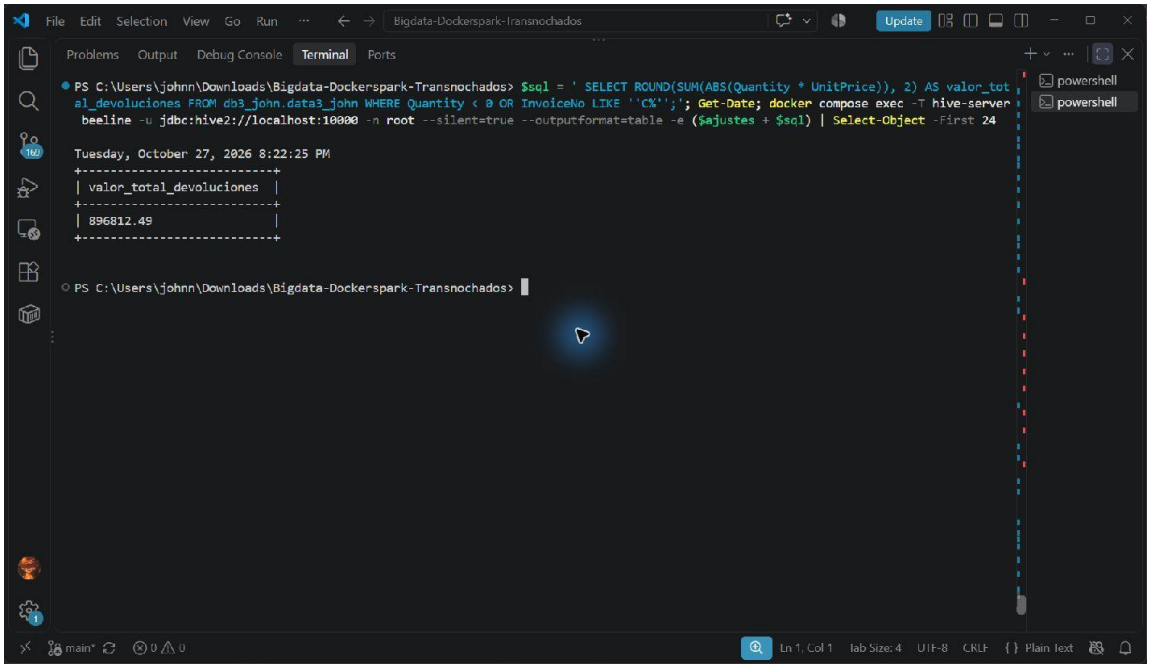
20	¿Cuántas devoluciones o cancelaciones existen?	SELECT COUNT(*) AS total_devoluciones FROM db3_john.data3_john WHERE Quantity < 0 OR InvoiceNo LIKE 'C%';	Cuenta registros asociados a devoluciones o cancelaciones. En este dataset, muchas cancelaciones aparecen con cantidades negativas o facturas que empiezan con C.



Evidencia 20. SQL y resultado en la terminal integrada de VS Code.

Respuesta y revisión de la descripción: Se contaron 10.624 registros con cantidad negativa o InvoiceNo que comienza por C. La consulta responde a líneas asociadas a devoluciones o cancelaciones; no cuenta documentos distintos ni unidades. También puede incluir ajustes de inventario con cantidad negativa.

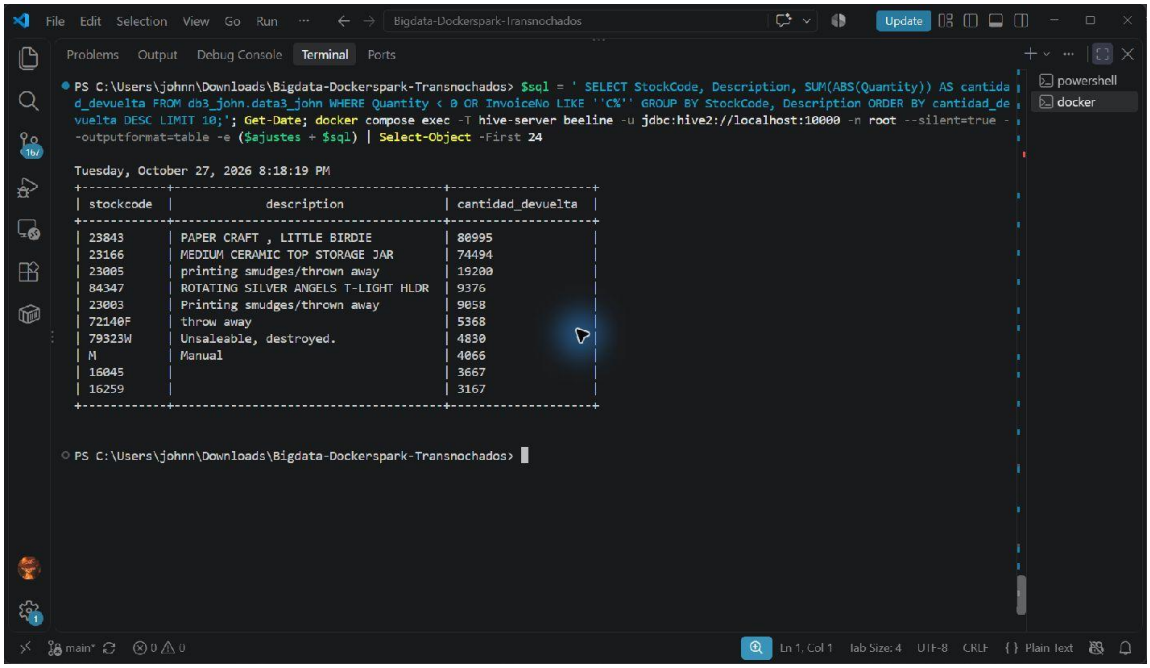
21	¿Cuál es el valor económico total de las devoluciones?	SELECT ROUND(SUM(ABS(Quantity * UnitPrice)), 2) AS valor_total_devoluciones FROM db3_john.data3_john WHERE Quantity < 0 OR InvoiceNo LIKE 'C%';	Calcula el valor monetario aproximado de los productos devueltos o cancelados. Es útil para evaluar pérdidas o ajustes comerciales.



Evidencia 21. SQL y resultado en la terminal integrada de VS Code.

Respuesta y revisión de la descripción: El valor absoluto de las líneas clasificadas como devolución fue de 896.812,49. ABS convierte el importe de cada línea a positivo antes de sumarlo. Es una medida de magnitud de devoluciones y ajustes, no una pérdida contable comprobada ni un cálculo de costos.

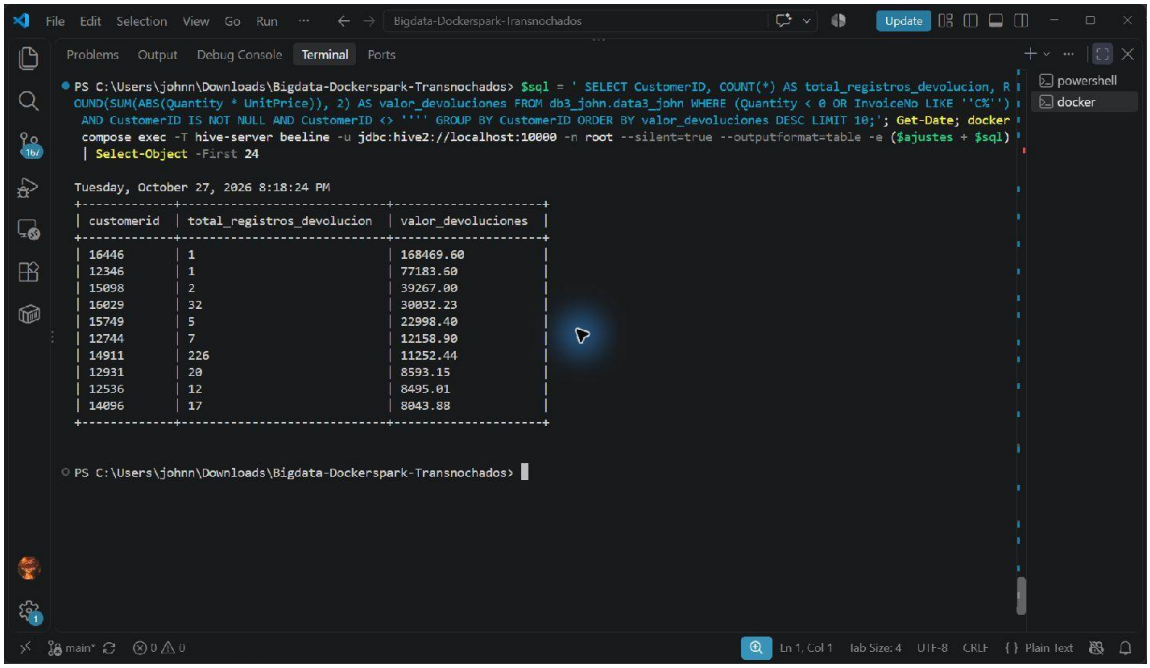
22	¿Qué productos tienen más devoluciones?	SELECT StockCode, Description, SUM(ABS(Quantity)) AS cantidad_devuelta FROM db3_john.data3_john WHERE Quantity < 0 OR InvoiceNo LIKE 'C%' GROUP BY StockCode, Description ORDER BY cantidad_devuelta DESC LIMIT 10;	Identifica los productos más devueltos. Puede ayudar a detectar problemas de calidad, errores de pedido o productos conflictivos.



Evidencia 22. SQL y resultado en la terminal integrada de VS Code.

Respuesta y revisión de la descripción: PAPER CRAFT, LITTLE BIRDIE, código 23843, encabezó las unidades devueltas con 80.995; MEDIUM CERAMIC TOP STORAGE JAR, código 23166, registró 74.494. La descripción printing smudges/thrown away acumuló 19.200 y muestra que también existen anotaciones de ajuste. El SQL suma unidades absolutas por código y descripción; no permite atribuir automáticamente el resultado a problemas de calidad.

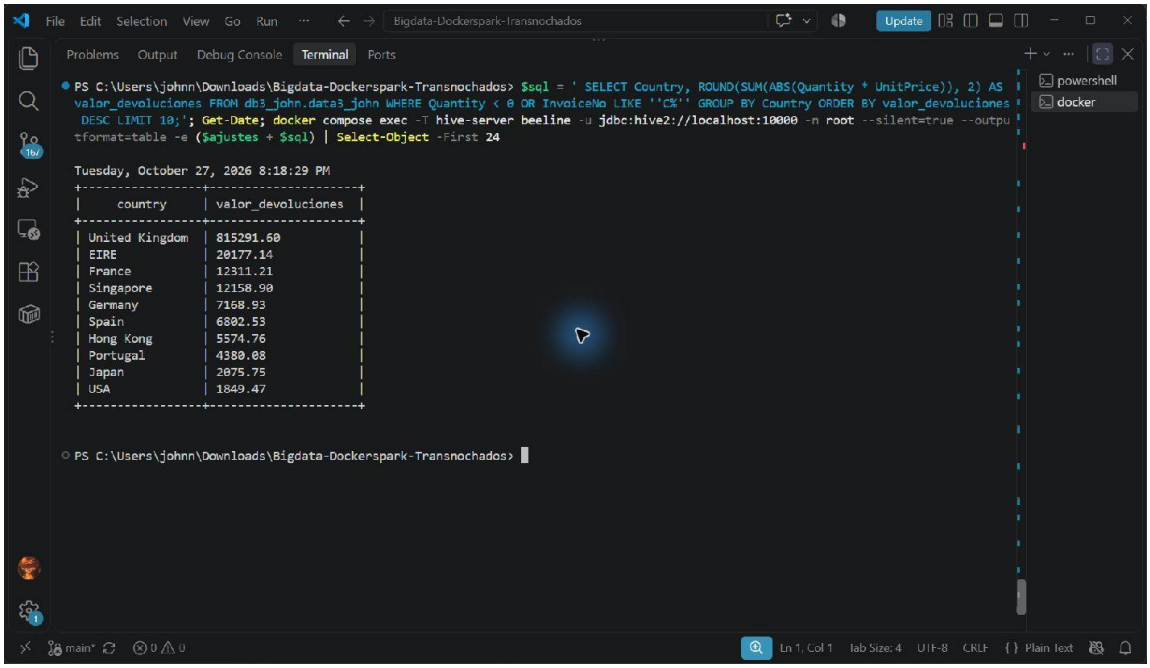
23	¿Qué clientes realizaron más devoluciones?	SELECT CustomerID, COUNT(*) AS total_registros_devolucion, ROUND(SUM(ABS(Quantity * UnitPrice)), 2) AS valor_devoluciones FROM db3_john.data3_john WHERE (Quantity < 0 OR InvoiceNo LIKE 'C%') AND CustomerID IS NOT NULL AND CustomerID <> '' GROUP BY CustomerID ORDER BY valor_devoluciones DESC LIMIT 10;	Muestra los clientes con mayor valor de devoluciones. Sirve para análisis de comportamiento del cliente y control comercial.



Evidencia 23. SQL y resultado en la terminal integrada de VS Code.

Respuesta y revisión de la descripción: El cliente 16446 presentó el mayor valor de devolución, 168.469,60 en una línea; 12346 registró 77.183,60 en una línea y 15098, 39.267,00 en dos. La consulta ordena por valor monetario, no por cantidad de devoluciones, aunque la pregunta utiliza la expresión más devoluciones. Ambas métricas aparecen separadas en el resultado.

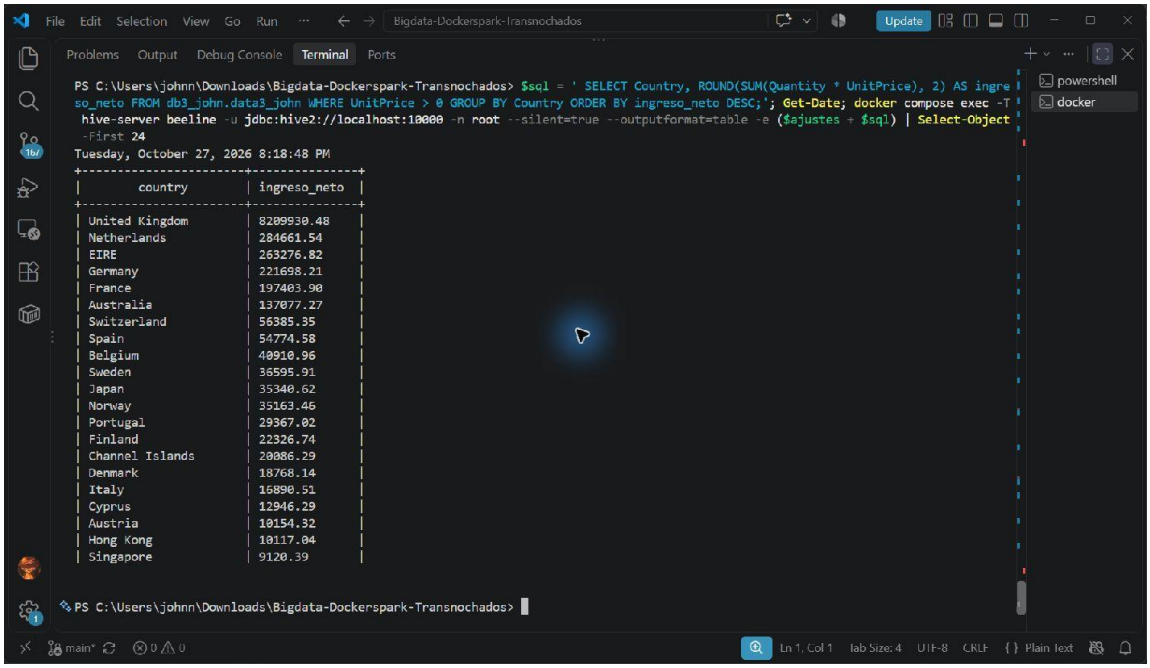
24	¿Cuál es el país con mayor valor de devoluciones?	SELECT Country, ROUND(SUM(ABS(Quantity * UnitPrice)), 2) AS valor_devoluciones FROM db3_john.data3_john WHERE Quantity < 0 OR InvoiceNo LIKE 'C%' GROUP BY Country ORDER BY valor_devoluciones DESC LIMIT 10;	Permite identificar los países donde se concentran más devoluciones en valor económico.



Evidencia 24. SQL y resultado en la terminal integrada de VS Code.

Respuesta y revisión de la descripción: United Kingdom concentró 815.291,60 de valor absoluto de devoluciones, seguido de EIRE con 20.177,14 y France con 12.311,21. El SQL responde al monto por país, no a una tasa relativa. Un país con más ventas puede acumular un importe mayor sin tener peor proporción de devoluciones.

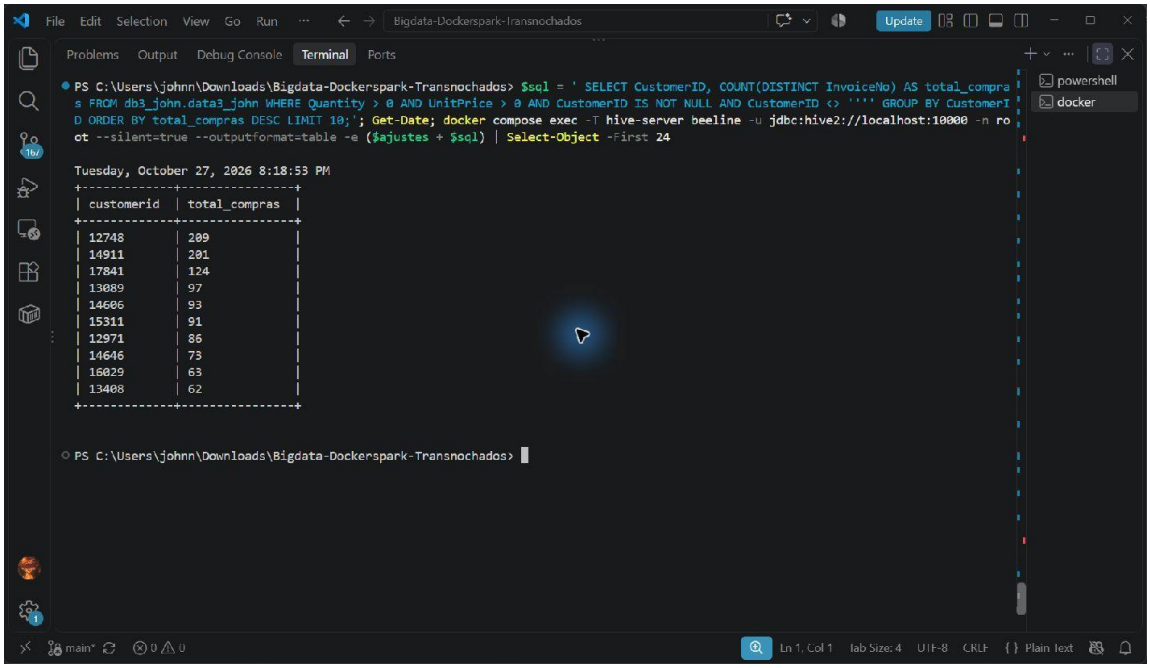
25	¿Cuál es el ingreso neto por país, considerando ventas y devoluciones?	SELECT Country, ROUND(SUM(Quantity * UnitPrice), 2) AS ingreso_neto FROM db3_john.data3_john WHERE UnitPrice > 0 GROUP BY Country ORDER BY ingreso_neto DESC;	Calcula el ingreso neto por país, ya que las cantidades negativas reducen el total. Es más realista que considerar solo ventas positivas.



Evidencia 25. SQL y resultado en la terminal integrada de VS Code.

Respuesta y revisión de la descripción: United Kingdom obtuvo 8.209.930,48 de ingreso neto bajo el filtro de precio positivo; Netherlands, 284.661,54; y EIRE, 263.276,82. La suma conserva cantidades negativas, que reducen los importes. Responde al saldo de ventas y ajustes incluidos, pero no equivale a utilidad neta ni contempla costos, impuestos o precios no positivos.

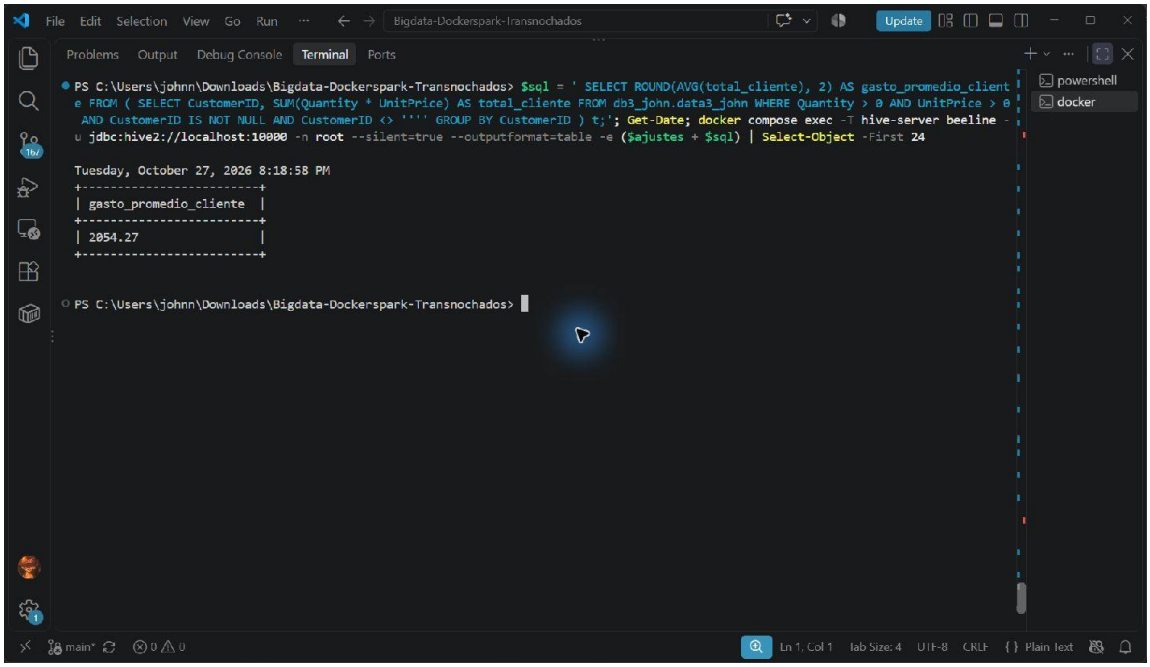
26	¿Qué clientes tienen mayor frecuencia de compra?	SELECT CustomerID, COUNT(DISTINCT InvoiceNo) AS total_compras FROM db3_john.data3_john WHERE Quantity > 0 AND UnitPrice > 0 AND CustomerID IS NOT NULL AND CustomerID <> '' GROUP BY CustomerID ORDER BY total_compras DESC LIMIT 10;	Identifica los clientes que compran más veces, sin importar el monto. Es útil para analizar fidelidad y recurrencia.



Evidencia 26. SQL y resultado en la terminal integrada de VS Code.

Respuesta y revisión de la descripción: El cliente 12748 tuvo 209 compras distintas; 14911, 201; y 17841, 124. Se cuentan facturas positivas por cliente identificado. La consulta responde a frecuencia durante el período, independientemente del importe; no mide antigüedad ni periodicidad entre compras.

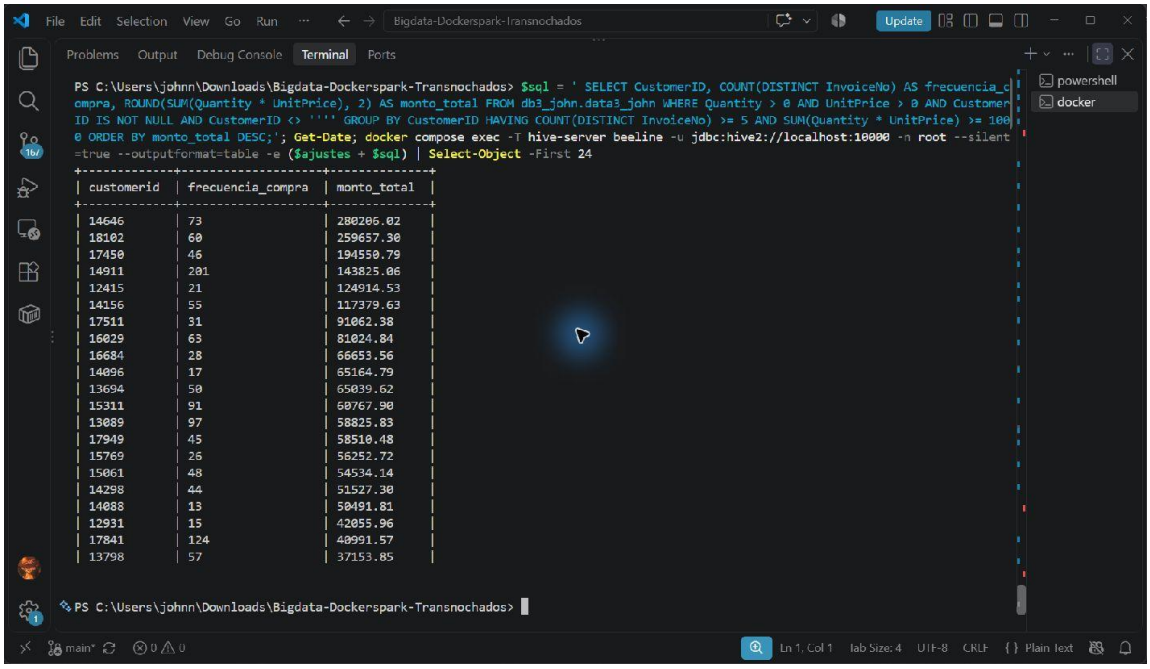
27	¿Cuál es el gasto promedio por cliente?	SELECT ROUND(AVG(total_cliente), 2) AS gasto_promedio_cliente FROM (SELECT CustomerID, SUM(Quantity * UnitPrice) AS total_cliente FROM db3_john.data3_john WHERE Quantity > 0 AND UnitPrice > 0 AND CustomerID IS NOT NULL AND CustomerID <> '' GROUP BY CustomerID) t;	Calcula cuánto gasta en promedio cada cliente durante el periodo analizado. Es un indicador comercial importante.



Evidencia 27. SQL y resultado en la terminal integrada de VS Code.

Respuesta y revisión de la descripción: El gasto promedio acumulado por cliente con compras válidas fue de 2.054,27. Primero se suman las ventas positivas de cada cliente y después se promedian esos totales. Es el gasto por cliente durante el período completo, no ticket por factura ni promedio mensual.

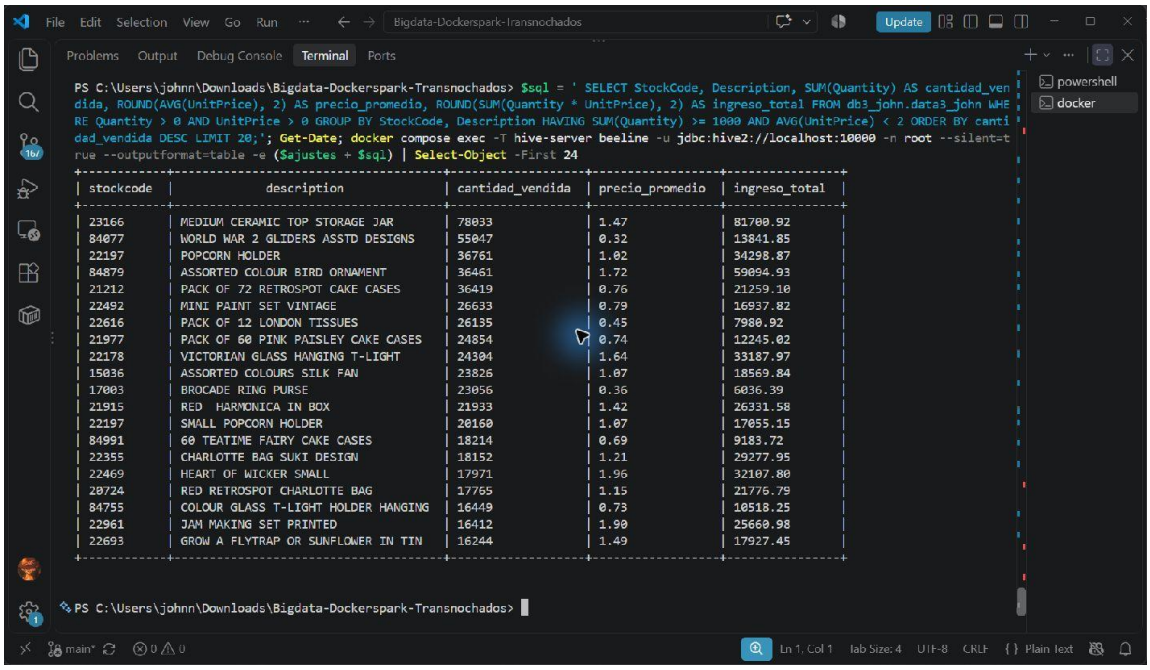
28	¿Qué clientes tienen alto valor y alta frecuencia de compra?	SELECT CustomerID, COUNT(DISTINCT InvoiceNo) AS frecuencia_compra, ROUND(SUM(Quantity * UnitPrice), 2) AS monto_total FROM db3_john.data3_john WHERE Quantity > 0 AND UnitPrice > 0 AND CustomerID IS NOT NULL AND CustomerID <> '' GROUP BY CustomerID HAVING COUNT(DISTINCT InvoiceNo) >= 5 AND SUM(Quantity * UnitPrice) >= 1000 ORDER BY monto_total DESC;	Identifica clientes importantes porque compran muchas veces y además generan alto ingreso. Sirve para segmentación tipo clientes VIP.



Evidencia 28. SQL y resultado en la terminal integrada de VS Code.

Respuesta y revisión de la descripción: Se identificaron 1.028 clientes con al menos cinco facturas positivas y un importe acumulado mínimo de 1.000. El cliente 14646 tuvo 73 compras y 280.206,02; 18102, 60 y 259.657,30; y 17450, 46 y 194.550,79. La consulta responde a los umbrales establecidos de frecuencia y valor; la etiqueta VIP depende de esos criterios, no es una categoría preexistente del dataset.

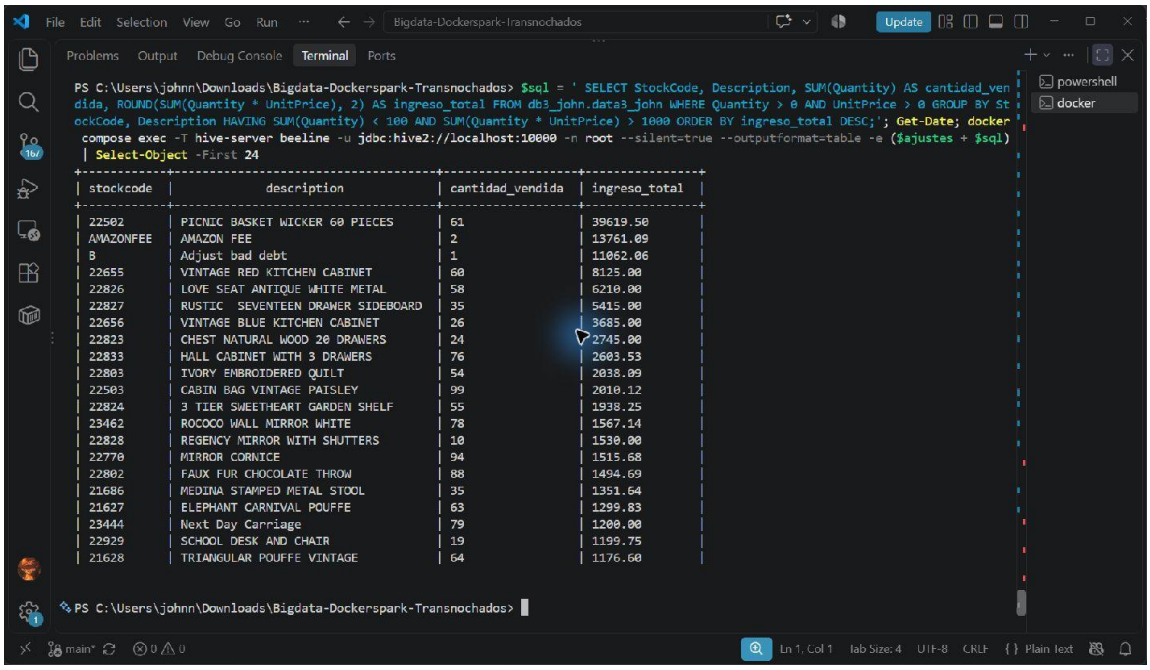
29	¿Qué productos tienen ventas altas pero bajo precio unitario?	SELECT StockCode, Description, SUM(Quantity) AS cantidad_vendida, ROUND(AVG(UnitPrice), 2) AS precio_promedio, ROUND(SUM(Quantity * UnitPrice), 2) AS ingreso_total FROM db3_john.data3_john WHERE Quantity > 0 AND UnitPrice > 0 GROUP BY StockCode, Description HAVING SUM(Quantity) >= 1000 AND AVG(UnitPrice) < 2 ORDER BY cantidad_vendida DESC LIMIT 20;	Encuentra productos económicos que se venden en grandes cantidades. Son productos de alta rotación.



Evidencia 29. SQL y resultado en la terminal integrada de VS Code.

Respuesta y revisión de la descripción: El código 23166, MEDIUM CERAMIC TOP STORAGE JAR, lideró con 78.033 unidades, precio promedio de 1,47 e ingreso de 81.700,92. WORLD WAR 2 GLIDERS ASSTD DESIGNS registró 55.047 unidades, precio medio de 0,32 e ingreso de 13.841,85. El filtro identifica grupos con al menos 1.000 unidades y precio promedio inferior a 2. La media de precios es simple por línea y no ponderada.

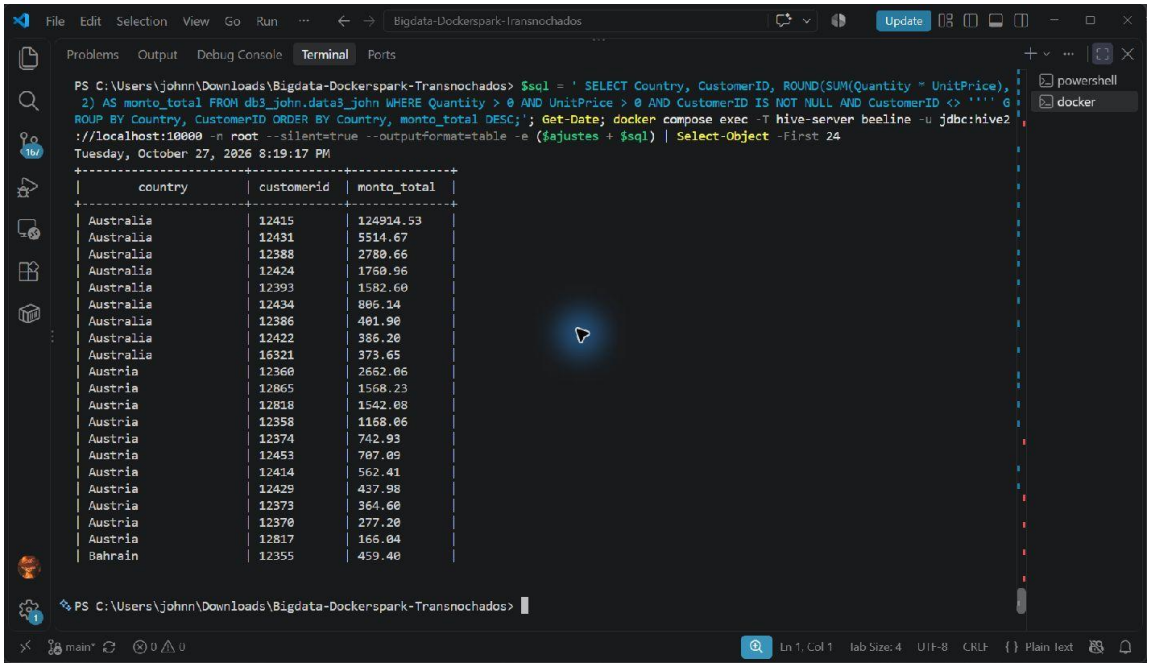
30	¿Qué productos tienen bajo volumen pero alto ingreso?	SELECT StockCode, Description, SUM(Quantity) AS cantidad_vendida, ROUND(SUM(Quantity * UnitPrice), 2) AS ingreso_total FROM db3_john.data3_john WHERE Quantity > 0 AND UnitPrice > 0 GROUP BY StockCode, Description HAVING SUM(Quantity) < 100 AND SUM(Quantity * UnitPrice) > 1000 ORDER BY ingreso_total DESC;	Identifica productos que no se venden tanto en cantidad, pero generan altos ingresos. Pueden ser productos premium o de alto margen.



Evidencia 30. SQL y resultado en la terminal integrada de VS Code.

Respuesta y revisión de la descripción: PICNIC BASKET WICKER 60 PIECES, código 22502, generó 39.619,50 con 61 unidades. AMAZONFEE registró 13.761,09 con dos unidades y Adjust bad debt, 11.062,06 con una. Se obtuvieron 22 grupos que cumplen menos de 100 unidades y más de 1.000 de ingreso. El SQL responde al contraste de volumen e importe, pero no demuestra un alto margen porque faltan costos y varios conceptos son cargos.

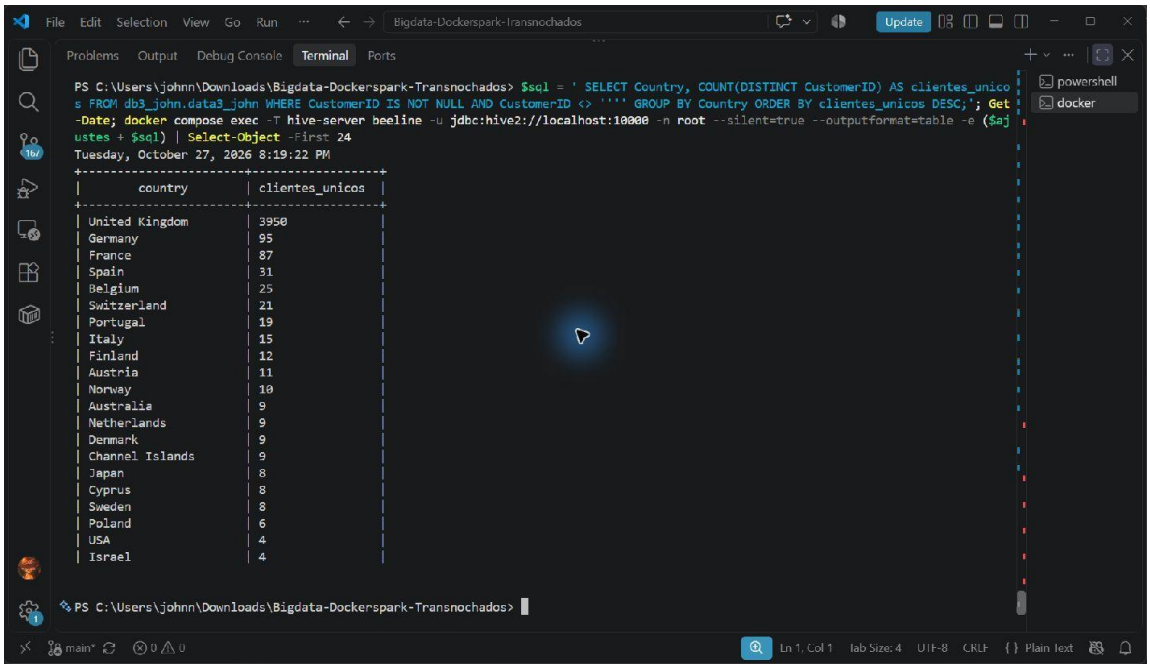
31	¿Cuál es el ranking de clientes por país según monto comprado?	SELECT Country, CustomerID, ROUND(SUM(Quantity * UnitPrice), 2) AS monto_total FROM db3_john.data3_john WHERE Quantity > 0 AND UnitPrice > 0 AND CustomerID IS NOT NULL AND CustomerID <> '' GROUP BY Country, CustomerID ORDER BY Country, monto_total DESC;	Muestra los clientes más importantes dentro de cada país. Sirve para análisis comercial por mercado.



Evidencia 31. SQL y resultado en la terminal integrada de VS Code.

Respuesta y revisión de la descripción: La consulta produjo 4.346 combinaciones de país y cliente con ventas positivas, ordenadas por país y monto descendente. En Australia, 12415 lideró con 124.914,53, seguido de 12431 con 5.514,67 y 12388 con 2.780,66. Responde a un orden comercial dentro de cada país; no genera una columna numérica de ranking y no debe interpretarse como un orden global de todos los clientes.

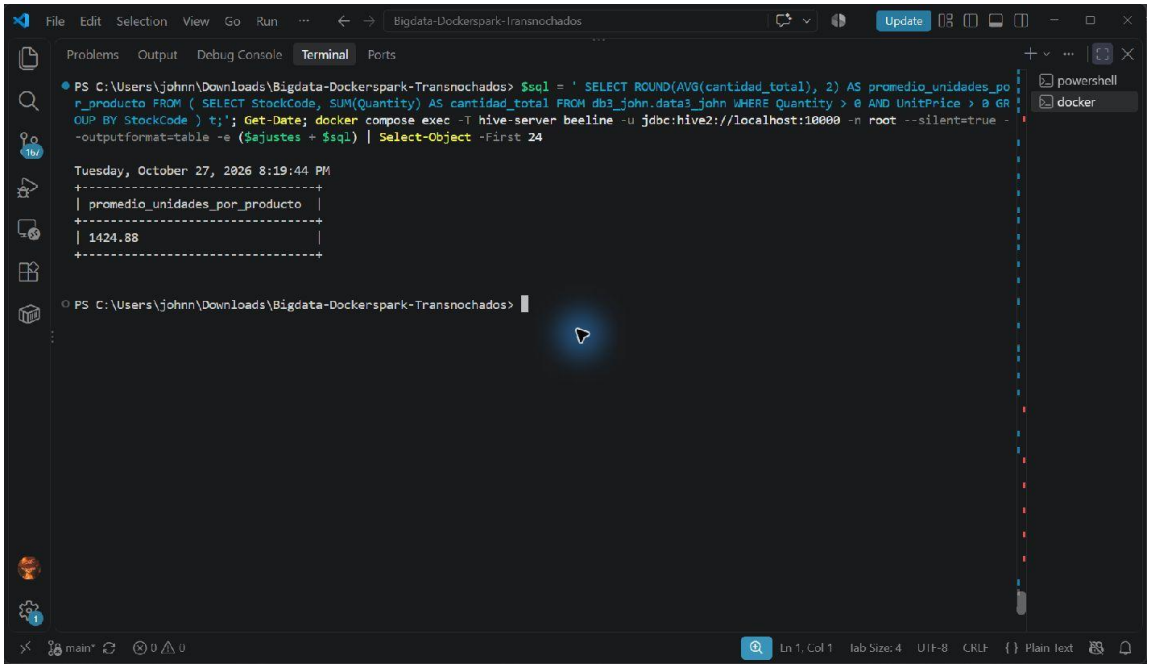
32	¿Qué países tienen mayor cantidad de clientes únicos?	SELECT Country, COUNT(DISTINCT CustomerID) AS clientes_unicos FROM db3_john.data3_john WHERE CustomerID IS NOT NULL AND CustomerID <> '' GROUP BY Country ORDER BY clientes_unicos DESC;	Identifica los países con mayor base de clientes. Es útil para estrategias comerciales y expansión internacional.



Evidencia 32. SQL y resultado en la terminal integrada de VS Code.

Respuesta y revisión de la descripción: United Kingdom presentó 3.950 clientes identificados distintos, Germany 95 y France 87. Se obtuvieron 37 categorías de país. El SQL responde a la base de clientes presentes en los registros, pero no exige ventas positivas: incluye clientes que solo pudieran figurar en devoluciones.

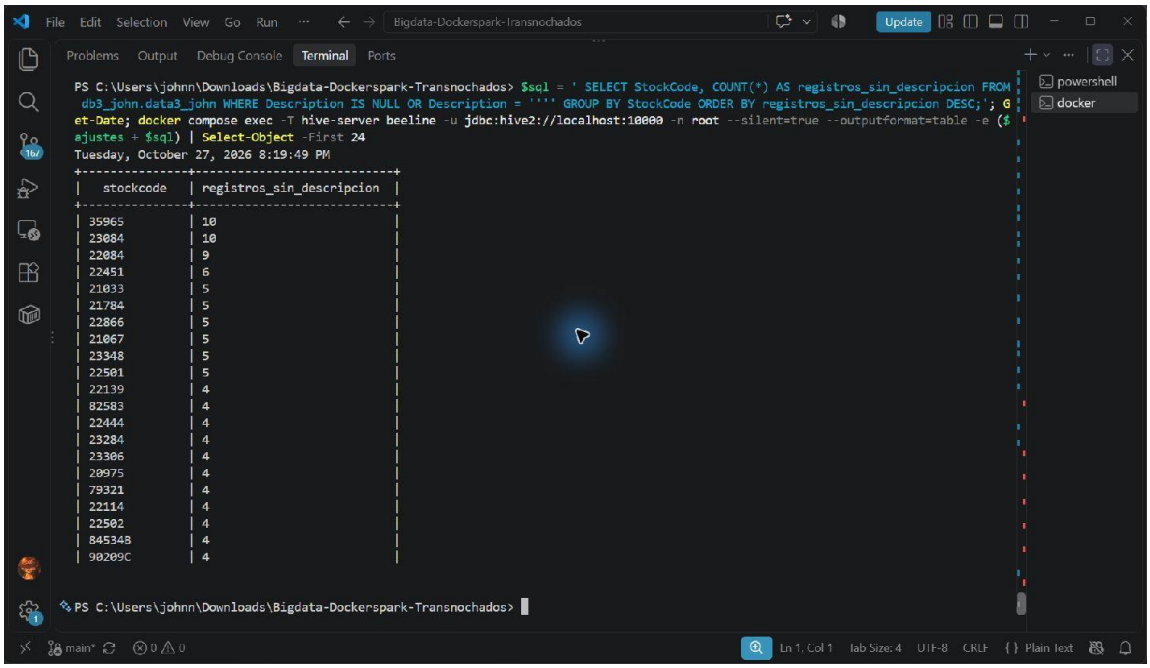
33	¿Cuál es el promedio de unidades vendidas por producto?	SELECT ROUND(AVG(cantidad_total), 2) AS promedio_unidades_por_producto FROM (SELECT StockCode, SUM(Quantity) AS cantidad_total FROM db3_john.data3_john WHERE Quantity > 0 AND UnitPrice > 0 GROUP BY StockCode) t;	Calcula el promedio de unidades vendidas por producto. Sirve para tener una referencia general del movimiento del inventario.



Evidencia 33. SQL y resultado en la terminal integrada de VS Code.

Respuesta y revisión de la descripción: El promedio de unidades positivas vendidas por código de producto fue de 1.424,88. La subconsulta suma Quantity por StockCode y la consulta exterior promedia esos totales. Responde al promedio por código, no a una media por línea ni por factura; tampoco descuenta devoluciones.

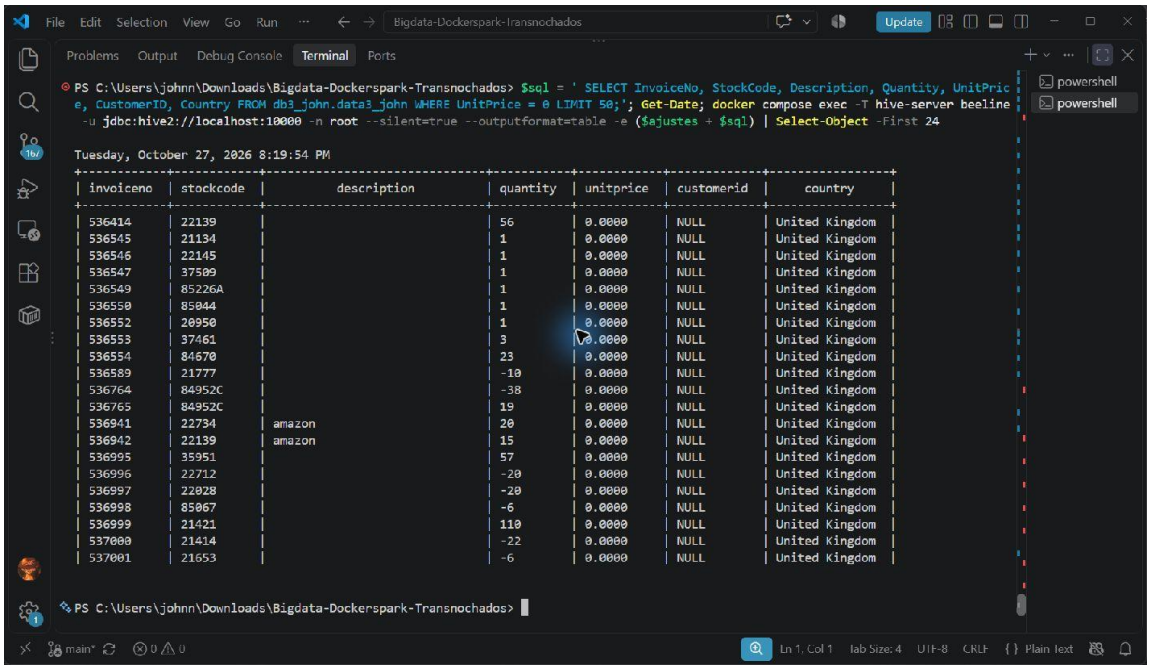
34	¿Qué productos no tienen descripción registrada?	SELECT StockCode, COUNT(*) AS registros_sin_descripcion FROM db3_john.data3_john WHERE Description IS NULL OR Description = '' GROUP BY StockCode ORDER BY registros_sin_descripcion DESC;	Detecta productos con problemas de calidad de datos. Es útil para limpieza y normalización del dataset.



Evidencia 34. SQL y resultado en la terminal integrada de VS Code.

Respuesta y revisión de la descripción: Se detectaron 960 códigos con al menos una línea sin descripción. Los códigos 35965 y 23084 tuvieron diez registros cada uno y 22084, nueve. La consulta cuenta filas vacías por código. No implica que esos códigos carezcan siempre de descripción: pueden tener otras líneas completas.

35	¿Qué registros tienen precio unitario igual a cero?	SELECT InvoiceNo, StockCode, Description, Quantity, UnitPrice, CustomerID, Country FROM db3_john.data3_john WHERE UnitPrice = 0 LIMIT 50;	Muestra registros con precio cero. Pueden representar promociones, errores de carga o registros que necesitan revisión.

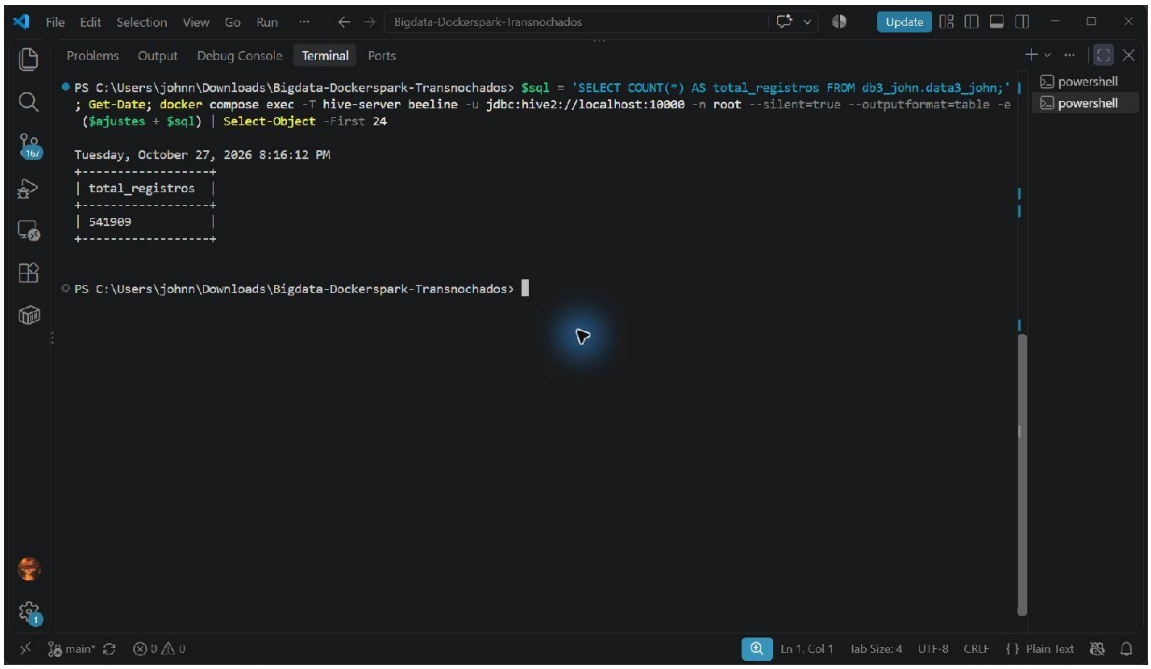


Evidencia 35. SQL y resultado en la terminal integrada de VS Code.

Respuesta y revisión de la descripción: Se mostraron 50 registros con UnitPrice igual a cero mediante LIMIT 50. Entre ellos figura la factura 536414, código 22139, con 56 unidades, sin descripción ni CustomerID y país United Kingdom. La consulta identifica ejemplos para revisión; no calcula el total de registros con precio cero ni permite decidir por sí sola si corresponden a promociones o errores.

El formato de respuesta esperado debe incluir la imagen de evidencia del ejecutado, asi como la respuesta en sus palabras, similar a:

Nro.	Pregunta	solución	Descripción de lo que se obtiene
1	Validación de la cantidad de registros	SELECT COUNT(*) FROM db3_john.data3_john;	La carga contiene 541.909 registros.



Respuesta a la pregunta: el dataset contiene 541.909 filas. El conteo de Hive coincide con los registros convertidos desde el Excel y respalda la validación de la carga.

Conclusiones

Se completaron la descarga del Excel, su conversión a CSV, la carga en HDFS y Hive y la ejecución de las 35 consultas. Las evidencias incluidas son capturas reales de la terminal integrada de VS Code con los comandos, la hora del sistema y la salida de Hive.

El dataset contiene 541.909 registros, 25.900 identificadores de factura y 4.372 clientes identificados en el conjunto completo. Las ventas positivas sumaron 10.666.684,54, el ticket medio por factura fue de 534,40 y el promedio de unidades por factura, 279,98. Estos indicadores utilizan poblaciones y filtros diferentes y se interpretaron de acuerdo con cada SQL.

Se contaron 10.624 líneas de devolución o ajuste y un valor absoluto asociado de 896.812,49. Su análisis permite contrastar ventas brutas y saldos netos, pero no equivale a pérdidas contables. La presencia de portes, comisiones y ajustes entre los códigos exige revisar la naturaleza de los conceptos antes de calificarlos como productos premium o de alto margen.

La práctica permitió diferenciar filas, facturas, unidades y clientes, así como ingresos, precios y utilidad. También mostró que agrupar por mes sin año mezcla períodos, que una consulta ordenada por monto no responde necesariamente a frecuencia y que los datos incompletos requieren revisión antes de tomar decisiones comerciales.

Para ejecutar las consultas se utilizó MapReduce local con mapreduce.task.io.sort.mb=10 y la ejecución vectorizada desactivada. Las vistas extensas se limitaron en la terminal mediante Select-Object, sin alterar los filtros ni las agregaciones de los SQL. Los comentarios del profesor se reservaron para su evaluación.